arXiv:2607.13816v2 [quant-ph] 9 Aug 2026

Quantum Algorithm for Elliptic Curve Discrete Logarithms with Space-Efficient Point Addition*

Han Luo[†1], Ziyi Yang[†2,3], Jingquan Luo[†4], Ziruo Wang[2,3], Yuexin Su[2,3], Xiaoming Sun[5,6], Lvzhou Li[‡4], and Tongyang Li[§2,3]

[1]Institute for Interdisciplinary Information Sciences, Tsinghua University
[2]Center on Frontiers of Computing Studies, Peking University
[3]School of Computer Science, Peking University
[4]Institute of Quantum Computing and Software, School of Computer Science and Engineering, Sun Yat-Sen University
[5]State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences
[6]School of Computer Science and Technology, University of Chinese Academy of Sciences

Abstract

The Elliptic Curve Discrete Logarithm Problem (ECDLP) is a fundamental problem in cryptography, and reducing the resource requirements of quantum algorithms for solving ECDLP is an important goal. In this work, we present a space-efficient quantum algorithm for solving the ECDLP over prime fields, achieving an implementation with only $3n + 6\lfloor \log_2 n \rfloor + O(1)$ logical qubits and $1008n^3/\log_2 n + O(n^2)$ Toffoli gates, where n is the bit-length of the prime. For a 256-bit prime-field curve, our construction requires only 835 logical qubits, reducing the previous best estimates of 1098 and 1175 logical qubits by Chevignard et al. [EUROCRYPT 2026] and Babbush et al. [ArXiv Preprint 2026], respectively.

The key to our improvement is a new space-efficient reversible modular inversion circuit, which addresses the dominant space bottleneck in affine-coordinate point addition. Starting from the extended Euclidean algorithm (EEA), we refine the register-sharing technique of Proos and Zalka by introducing length registers and location-controlled arithmetic to compactly store and update intermediate variables. We further optimize the reversible update procedures and construct the corresponding controlled arithmetic circuits, resulting in a modular inversion circuit implemented by only $2n + 6\lfloor \log_2 n \rfloor + O(1)$ logical qubits and $217n^2 + O(n \log_2 n)$ Toffoli gates. This modular inversion circuit together with mid-circuit measurements and classical feed-forward operations provides a space-efficient controlled affine point-addition circuit and a complete implementation of Shor's algorithm for ECDLP.

*This paper supersedes our earlier preprint arXiv:2604.02311. Compared with the earlier version, the present paper reduces the space complexity from $5n + O(\log_2 n)$ to $3n + O(\log_2 n)$ for affine point addition and from $3n + O(\log_2 n)$ to $2n + O(\log_2 n)$ for modular inversion. It also improves the overall Toffoli counts of ECDLP from $O(n^3)$ to $O(n^3/\log n)$.

†Equal contribution.
‡Corresponding author. Email: lilvzh@mail.sysu.edu.cn
§Corresponding author. Email: tongyangli@pku.edu.cn

1 Introduction

Elliptic curve cryptography (ECC), first proposed in the mid-1980s as an alternative framework for public-key cryptography [Kob87, Mil85], has become one of the foundational primitives in modern cryptographic systems. The security of ECC-based cryptosystems relies on the computational difficulty of the Elliptic Curve Discrete Logarithm Problem (ECDLP). Specifically, given an elliptic curve defined over a finite field, a base point P on that curve, and another point Q resulting from scalar multiplication of P by an unknown integer m, the ECDLP asks for the recovery of the scalar m from the two points. Despite decades of research, no classical algorithm is known that solves the ECDLP in polynomial time of bit length; the best generic attacks, such as Pollard's rho method, require $\Theta(\sqrt{p})$ group operations, where p denotes the field size of the curve which is exponentially large [GG16].

Compared to classical public-key systems such as RSA [RSA78] and finite-field Diffie-Hellman [DH76], ECC offers equivalent security with substantially smaller key sizes. For example, according to NIST recommendations [BD20], a 256-bit elliptic curve provides a security level of 128 bits, which is comparable to that of RSA with a 3072-bit modulus. This efficiency advantage has led to the widespread adoption of ECC in both theoretical cryptographic constructions and real-world applications. Specifically, ECC is used for key exchange [DH76] and digital signature schemes [ElG85, JMV01] in widely deployed protocols, including transport layer security [BWBG+06], secure shell [SG09], as well as cryptocurrencies such as Bitcoin [BWQ99, NB08].

In contrast to the computational hardness of ECDLP on classical computers, the security landscape changes fundamentally in the presence of large-scale quantum computers. In 1994, Shor gave polynomial-time quantum algorithms for integer factorization and discrete logarithms [Sho94]. Because the ECDLP is a discrete-logarithm problem in a cyclic subgroup of an elliptic-curve group, it can be efficiently solved quantumly by Shor's algorithm whenever the elliptic-curve group operation can be implemented efficiently. Proos and Zalka subsequently developed an implementation with resource analysis for prime-field elliptic curves [PZ03]. Thus, sufficiently large fault-tolerant quantum computers would break conventional ECC based on the hardness of the ECDLP. At the same time, the practical implementation of quantum algorithms for cryptography problems remains severely constrained by the limitations of near-term quantum hardware. For instance, implementing Shor's algorithm at cryptographically relevant scales requires a fault-tolerant quantum computer with a large number of logical qubits and circuits, as well as sufficiently low physical error rates (or equivalently, achievable logical error rates after error correction) [GE21, Gid25].

Motivated by these challenges, a substantial body of work has focused on reducing the quantum resource requirements of Shor's algorithm for integer factorization [Reg25, RV24, KMRVvK25, CFS25] and the ECDLP [RNSL17, HJN+20, GYCM25, KJW+26, CFS26, BZG+26, Sch26]. In recent studies, three primary resource metrics have received the most attention: (i) space, measured by the number of logical qubits; (ii) quantum gate complexity; and (iii) circuit depth. These resources are strongly coupled, and optimizing one metric often leads to increased costs in others [RNSL17, HJN+20, BZG+26, Sch26]. For each single metric, the state-of-the-art results are:

- In terms of space complexity, Chevignard, Fouque, and Schrottenloher [CFS26] proposed a low-space

variant of Shor's ECDLP algorithm based on residue-number-system arithmetic and compressed projective-coordinate outputs. Their construction achieves the state-of-the-art asymptotic qubit complexity among existing implementations, requiring only $3.12n + O(\sqrt{n})$ logical qubits for an n-bit prime field. However, this reduction in space comes at the cost of a sufficiently larger gate count of $\widetilde{O}(n^4)$.

- In terms of gate complexity, Babbush et al. [BZG+26] reported substantially improved resource estimates for solving ECDLP of the curve secp256k1 using 1191 logical qubits and $2^{26.27}$ Toffoli gates in the space-optimized setting, or 1441 logical qubits and $2^{25.94}$ Toffoli gates in the gate-optimized setting. However, their circuit-level details are not publicly disclosed. Schrottenloher [Sch26] subsequently gave an explicit and reproducible point addition architecture with comparable performance and slightly smaller Toffoli counts for secp256k1.

- In terms of circuit depth, Kim et al. [KJW+26] developed depth-optimized point-addition circuits based on QCSA-based Montgomery multiplication and low-depth binary-EEA inversion. They achieved the lowest circuit depth with adequate ancilla qubits. In terms of the product of qubit count and full depth, their qubit-optimized version achieves a 37%–40% improvement compared to the previous result by Häner et al. [HJN+20].

In this work, we study the space complexity of ECDLP. In particular, our focus is the modular inversion subroutine, which is the dominant source of workspace in affine-coordinate point addition. Our goal is to answer the following question:

Can the quantum space requirements for solving the ECDLP be further reduced in practice?

1.1 Contributions

The idea of reducing the space complexity of quantum ECDLP algorithms can be traced back to Proos and Zalka [PZ03], who introduced a register-sharing strategy and obtained an asymptotic space bound of $5n + O(\sqrt{n})$ for solving the ECDLP over an n-bit prime field. However, their analysis was given at a high level of abstraction and did not provide explicit reversible circuit constructions or detailed resource estimates; in particular, issues such as reversible implementation and ancilla management were not fully addressed, and the approximate treatment of modular inversion introduces a fidelity loss of $O(n^{-1})$.

Our main contribution is to turn this low-space strategy into an exact and fully explicit circuit-level construction. Since modular inversion is the dominant contributor to the overall space, we focus on designing a more space-efficient reversible implementation of this component. Building on the algorithmic framework and register-sharing idea of Proos and Zalka, we refine the representation of the EEA state, introduce length registers and location-controlled arithmetic, and provide explicit circuit constructions with detailed resource estimates. As summarized in Table 1, our construction achieves the best asymptotic logical-qubit complexity reported so far for ECDLP in the standard quantum circuit model.

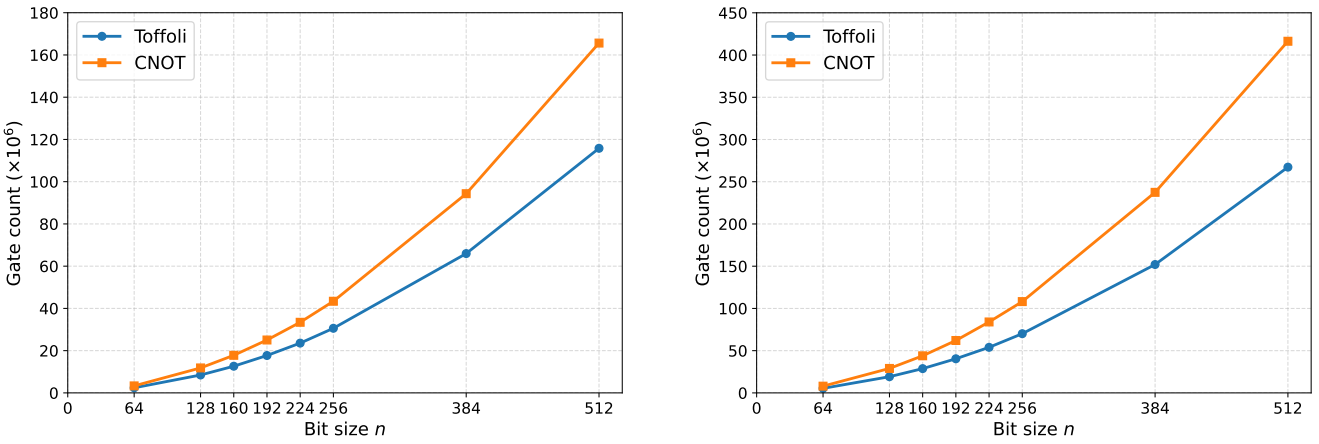
Second, we provide resource estimates for our quantum circuit implementation. Specifically, we show that a modular inversion circuit can be implemented using at most $217n^2 + O(n \log_2 n)$ Toffoli gates and $274n^2 + O(n \log_2 n)$ CNOT gates. By integrating this modular inversion circuit into an affine point

Reference	Space of point addition	Space of modular inversion
[RNSL17]	$9n + 2\log_2 n + O(1)$	$7n + 2\log_2 n + O(1)$
[HJN+20]	$8n + 10.2\log_2 n + O(1)$	$7n + \log_2 n + O(1)$
[CFS26]	$3.12n + o(n)$	not based on modular inversion
[Sch26]	$4.12n + O(\sqrt{n})$	$2.12n + O(\sqrt{n})$
This work	$3n + 6\log_2 n + O(1)$	$2n + 6\log_2 n + O(1)$

Table 1: Comparison of the number of logical qubits required for ECDLP and modular inversion.

addition circuit with mid-circuit measurement (Figures 15 and 14), we obtain a complexity of $1003n^2 + O(n\log_2 n)$ Toffoli gates per point addition. Furthermore, incorporating the point addition circuit into the complete quantum algorithm for solving the ECDLP using the signed-window technique [HJN+20, Section 5.1] yields a total Toffoli-gate complexity of $1008n^3/\log_2 n + O(n^2)$.

We complement the asymptotic comparison with numerical gate-count estimates and concrete secp256k1 resource estimates. Figure 1 reports the Toffoli and CNOT counts of our modular-inversion and point-addition circuits as functions of the field size n and Table 2 compares the resulting secp256k1 estimates with recent quantum ECDLP implementations. For secp256k1, our construction uses 835 logical qubits, saving more than 250 logical qubits compared to 1098 qubits in the previous best space-optimized estimate of Chevignard et al. [CFS25]. Furthermore, our Toffoli count is two orders of magnitude lower than [CFS25].



(a) modular inversion count

(b) point addition count

Figure 1: Toffoli gate count and CNOT gate count for modular inversion and point addition.

1.2 Overview

We present an overview of our quantum circuit for solving ECDLP. The standard implementation of Shor's algorithm consists of $2n+2$ controlled elliptic-curve point additions followed by a quantum Fourier transform (QFT), as illustrated in Figure 2. Replacing the QFT with its semiclassical variant reduces the logical-qubit requirement without altering the algorithmic functionality, as shown in Figure 3.

Reference	Type	Qubits	Toffolis
[BZG+26]	Space-optimized for secp256k1	1175	$2^{26.27}$
	Gate-optimized for secp256k1	1425	$2^{25.94}$
[Sch26]	Space-optimized for secp256k1	1192	$2^{26.11}$
	Gate-optimized for secp256k1	1446	$2^{25.78}$
[CFS26]	Space-optimized for secp256k1	1098	$2^{38.10}$
This work	Space-optimized for secp256k1	835	$2^{30.88}$

Table 2: Comparison of the concrete costs of circuits for secp256k1.

Each elliptic-curve point addition is decomposed into reversible modular arithmetic operations, including modular addition, multiplication, squaring, and division, etc. Among these, modular division is the dominant contributor to the logical-qubit cost. Figure 14 illustrates the structure of our point-addition circuit, where these operations are encapsulated as modular arithmetic blocks. In particular, the in-place modular division block is realized by the circuit shown in Figure 15, which consists of a forward execution of the extended Euclidean algorithm (EEA) for modular inversion, followed by its uncomputation.

We present each component from a top-down perspective as follows.

Space-efficient reversible algorithm for modular inversion (Section 3). Our modular inversion procedure, based on the EEA, is designed to implement the following quantum transformation:

$$|x\rangle_X|0\rangle_A|0\rangle_S \longmapsto |x^{-1} \bmod p\rangle_X|0\rangle_A|\Gamma(x)\rangle_S, \qquad x \in \mathbb{F}_p^\times, \tag{1}$$

where A is an n-qubit workspace register, S denotes the remaining ancillary registers, and $|\Gamma(x)\rangle$ is the forward EEA state retained for later uncomputation.

We first revisit the four-phase framework of Proos and Zalka [PZ03] (see Section 3.1), which is well suited for quantum implementation because the number of iterations (which we refer to as steps) is linear in n. Rather than relying on the (inaccurate) $4.5n$ upper bound stated in [PZ03], we establish a rigorous bound showing that a complete execution of the EEA requires at most $4\lceil cn\rceil$ steps for every input x, where $c = 3/\log_2(2+\sqrt{3})$. Specifically, we consider a different roadmap: instead of computing the upper bound of step number N, we fix the total number of steps and then compute the minimum possible value of p. To do this, we formulate the problem as finding a slowest growth pattern for products of matrices $M(q)$ with a fixed weight sum $N/4 = \sum_{i=1}^{m}(d(q_i)+1)$. Motivated by the asymptotically extremal alternating quotient pattern $(1, 2, 1, 2, \dots)$ whose normalized growth rate is $(2+\sqrt{3})^{1/3}$ per unit weight, we construct a finite polyhedral antinorm certificate, following the lower spectral radius framework of [GP13], to prove that every admissible quotient sequence grows at least this rapidly. Details can be found in Appendix A.1.

Section 3.2 then develops an explicit and refined register-sharing strategy at the algorithmic level. In the original construction of [PZ03], a uniform register allocation is used to accommodate all inputs $x \in \mathbb{F}_p$ in superposition. This worst-case allocation incurs an $O(\sqrt{n})$ space overhead and necessitates truncating rare outlier cases, introducing an $O(n^{-2})$ fidelity loss. In contrast, we present an exact allocation scheme

that guarantees a deterministic register bound for all inputs $x \in \mathbb{F}_p$. This reduces the additional space overhead to $O(\log n)$ while preserving exact correctness.

Finally, starting from a baseline reversible realization of this space-efficient EEA, we introduce a sequence of algorithmic optimizations in Section 3.3 that reorganize the update rules and eliminate redundant arithmetic operations. We provide complete pseudocode (Algorithm 3) together with representative classical execution traces (Table 4), which illustrate the reversibility and correctness of each step and clarify how the procedure can be translated into a quantum circuit.

Explicit quantum circuit construction for modular inversion (Section 4). To show that our modular inversion algorithm admits an explicit quantum circuit implementation, we present a detailed construction based on Algorithm 3. We begin with the overall circuit architecture (Figures 5 and 6 in Section 4.1), where location-controlled operations arise as key non-standard components.

Before giving their explicit implementations, we establish the necessary tools. In Section 4.2, we introduce step-dependent active windows, which restrict the relevant index range at each step and thereby reduce gate complexity. We then introduce unary iteration (Section 4.3) as a primitive for implementing controlled indexed operations, which enables the realization of these location-controlled components.

Finally, in Section 4.4, we present explicit constructions of the location-controlled circuit blocks, including swap, addition/subtraction, and length-update operations (Figures 9, 10, 12, and 13). In addition to the circuit diagrams, we provide detailed explanations of the underlying techniques and design rationale.

Affine point addition with mid-circuit measurements (Section 5). We further show that affine point addition can be implemented using our EEA inversion circuit with only $3n + O(\log n)$ logical qubits by allowing mid-circuit measurements and classical feed-forward operations. The main issue is the in-place division step

$$|x\rangle_X |y\rangle_Y |0\rangle_A \longmapsto |x\rangle_X |y/x\rangle_Y |0\rangle_A.$$

To perform this map, the EEA inversion circuit in Equation (1) is first applied to x. This requires one clean n-qubit register A and leaves the forward EEA state $\Gamma(x)$ to be cleared later. After the quotient y/x has been written into A, however, the register A is no longer clean, so the inverse EEA circuit has no available n-qubit workspace.

As shown in Figure 15, we resolve this problem by measuring the register containing the old value y in the Hadamard basis and resetting it to zero. This makes the register Y available as the clean workspace for the inverse EEA circuit, which recovers x and clears $\Gamma(x)$. This measurement is valid because, after y/x has been written into A, the old value y is redundant and can be recovered from x and y/x. Measuring $|y\rangle$ in the Hadamard basis gives a uniformly random outcome $b \in \{0,1\}^n$, and the only effect on the remaining registers is the phase $(-1)^{b \cdot y}$. Thus measuring the register Y does not destroy coherence. The phase is removed by recomputing y from x and y/x, applying the corresponding Pauli Z corrections, and uncomputing the recomputation. The same measurement and correction procedure gives an in-place multiplication circuit. Consequently, affine point addition can be implemented using three n-qubit registers X, Y, and A, together with the $O(\log n)$ auxiliary qubits required by the EEA

circuit.

We note that mid-circuit measurements have previously been used to reduce either the qubit count or the T-gate cost of quantum arithmetic circuits [Gid19, KSS25, LMNS25, KH21]. In particular, the lowest-qubit quantum algorithm currently known for the elliptic-curve discrete logarithm problem also makes extensive use of measurements to reduce the number of qubits [CFS26]. That work does not keep the full elliptic-curve point, nor does it reduce the index registers using a semiclassical Fourier transform. Instead of computing the full point, it computes a one-bit Legendre symbol from the projective coordinates of that point using residue-number-system arithmetic. Its measurements are used to release intermediate nodes in a spooky pebbling strategy for a binary tree [Gid19, KSS25]. In contrast, we keep the standard affine point-addition structure and use measurements only inside the in-place division and multiplication blocks, where they recycle a register needed as workspace for running the inverse EEA circuit.

Resource estimations (Section 6). Putting everything together, we quantify the space complexity and gate costs of the complete construction of Shor's ECDLP circuit. We first show in Section 6.1 that the modular inversion circuit requires $2n+6\lfloor\log_2 n\rfloor+19$ logical qubits, and that the affine point addition circuit of Section 5 can therefore be implemented using $3n+6\lfloor\log_2 n\rfloor+19$ logical qubits. Section 6.2 then combines the per-block gate costs (Section 4.4) with the step-dependent active-window bounds (Section 4.2), to obtain an upper bound of $217n^2+O(n\log_2 n)$ Toffoli gates and $274n^2+O(n\log_2 n)$ CNOT gates for modular inversion, as well as an upper bound of $1003n^2+O(n\log_2 n)$ Toffoli gates for one affine point addition.

We complement these asymptotic bounds with concrete circuit generation and blockwise compilation using Qiskit in Section 6.3, reporting numerical Toffoli and CNOT gate counts for prime-field sizes ranging from 64 to 512 bits. Finally, in Section 6.4, we incorporate signed-window scalar multiplication and the associated table look-ups into the complete Shor's ECDLP circuit. This yields an overall Toffoli complexity of $1008n^3/\log_2 n+O(n^2)$. For the `secp256k1` curve, our construction requires 835 logical qubits and $2^{30.88}$ Toffoli gates.

1.3 Open questions

Our work leaves several open questions for future investigation:

- **Gate-count optimization.** Our construction prioritizes logical-qubit efficiency and gives explicit reversible circuits for all arithmetic components. However, several building blocks, especially location-controlled arithmetic, length-register updates, and controlled modular operations, may still admit substantial improvements of their gate counts.
- **Depth analysis and space-depth trade-offs.** This paper focuses primarily on the number of logical qubits and Toffoli/CNOT gate counts, and does not provide a detailed analysis of circuit depth. A natural next step is to study the depth of our modular inversion and point-addition circuits, identify which parts of the computation can be parallelized, and quantify the resulting trade-offs between space, gate count, and depth.

- **Applications beyond elliptic-curve discrete logarithms.** We note that modular inversion is a general-purpose arithmetic primitive, not only restricted to ECDLP. It would be useful to investigate whether the space-efficient reversible EEA and register-sharing techniques developed here can improve other quantum algorithms that require modular division or inversion, such as Decoded Quantum Interferometry (DQI) [KSG+25].

- **AI-assisted discovery of quantum arithmetic circuits.** An important open question is whether AI systems can enable the automated design of quantum arithmetic circuits, supporting both the discovery of new resource-efficient circuits and the optimization of existing ones. One possible approach is to encode techniques such as windowing, unary iteration, register sharing and mid-circuit measurement as composable modules, and to develop AI-assisted evolutionary frameworks, inspired by systems such as AlphaEvolve [NV+25] and SimpleTES [YLT+26], that iteratively explore their replacement and composition under automated correctness verification and resource-aware evaluation.

2 Preliminaries

This section provides a very brief discussion of the basic concepts used in this work. We assume that readers have basic knowledge of quantum computation. For a general survey of quantum algorithms for algebraic problems, we recommend the survey written by Childs and van Dam [CvD10].

2.1 Quantum computing and the Toffoli counts

We write basis quantum states as $|x\rangle$, where x is a bit string, and we model a quantum algorithm as a circuit composed of elementary gates applied to one or more qubits. A key feature is that circuits can create and transform superpositions of basis states, enabling interference effects that are exploited by quantum algorithms.

At the fault-tolerant logical level, it is common to express circuits over a universal gate set such as Clifford+T [NC10]. In many architectures the T gate is substantially more expensive than Clifford operations, so circuit cost is frequently summarized by T-count, T-depth, or related metrics [FMMC12]. For design and validation, however, it is often convenient to use a purely reversible gate basis.

Accordingly, we express the main reversible components as *Toffoli networks*. The Toffoli gate maps

$$|x, y, z\rangle \longmapsto |x, y, z \oplus (xy)\rangle,$$

and it forms a universal primitive for classical reversible computation [NC10]. A practical benefit of working at the Toffoli-network level is that such networks admit exact realizations over Clifford+T [AMMR13], avoiding approximation overhead associated with synthesizing arbitrary unitaries. Moreover, Toffoli-based reversible circuits can be efficiently simulated on classical inputs at scales far beyond what is possible for general quantum-state simulation, which makes testing and debugging large arithmetic circuits significantly more tractable [HRS17]. For these reasons, we specify the core arithmetic and elliptic-curve routines using Toffoli and CNOT gates.

2.2 Elliptic curves and the ECDLP

Let p be a large prime and let $\mathbb{F}_p$ denote the finite field with p elements. An elliptic curve $E/\mathbb{F}_p$ (in short Weierstrass form) is the set of affine solutions $(x, y) \in \mathbb{F}_p^2$ to an equation $y^2 = x^3 + ax + b$, together with a distinguished point $\mathcal{O}$ at infinity. The set of $\mathbb{F}_p$-rational points, together with $\mathcal{O}$, is denoted by $E(\mathbb{F}_p)$.

The points on $E(\mathbb{F}_p)$ form an abelian group under the elliptic-curve addition law, with identity element $\mathcal{O}$. For affine points $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$ that are not inverse to each other, the sum $P_3 = P_1 + P_2$ can be computed via a slope parameter λ:

$$x_3 = \lambda^2 - x_1 - x_2, \qquad y_3 = \lambda(x_1 - x_3) - y_1,$$

where λ equals the chord slope $\frac{y_2 - y_1}{x_2 - x_1}$ when $P_1 \neq P_2$, and the tangent slope $\frac{3x_1^2 + a}{2y_1}$ when $P_1 = P_2$. Exceptional cases (e.g., $P_i = \mathcal{O}$ or $P_2 = -P_1$) are handled according to the standard group properties.

For an integer $m \geq 1$, the notation $[m]P$ denotes the m-fold sum of a point P with itself, i.e. $[m]P = P + P + \cdots + P$, where P occurs m times. This m-fold sum can be extended to all $m \in \mathbb{Z}$ by defining $[0]P = \mathcal{O}$ and $[-m]P = [m](-P)$ for $m \geq 1$. The operation $m \mapsto [m]P$ is called *scalar multiplication* and is the core primitive used in elliptic-curve cryptography.

Let $P \in E(\mathbb{F}_p)$ generates a cyclic subgroup $\langle P \rangle$ of order r, and let $Q \in \langle P \rangle$. The *elliptic-curve discrete logarithm problem (ECDLP)* is to recover the unique $m \in \{0, 1, \ldots, r-1\}$ such that

$$Q = [m]P.$$

In classical settings, the best generic algorithms require on the order of $\Theta(\sqrt{p})$ group operations [GG16], which is exponential in the bit length $\log_2 p$.

2.3 Shor's quantum algorithm for ECDLP

Shor's discrete-logarithm algorithm for solving ECDLP applies to elliptic curve over any finite abelian group and therefore to $E(\mathbb{F}_p)$. Let $n = \lfloor \log_2 p \rfloor + 1$ be the bit-length of p. The quantum procedure uses two exponent registers and one register storing an elliptic curve point.

Firstly, initialize two $(n+1)$-qubit registers[1] to be $|0^{n+1}, 0^{n+1}\rangle$, and apply Hadamard gate to all the qubits to obtain a uniform superposition over pairs $(k, \ell) \in \{0, 1, \ldots, 2^{n+1} - 1\}^2$. Next, coherently compute the elliptic curve group action into an accumulator as

$$\sum_{k,\ell=0}^{2^{n+1}-1} |k, \ell\rangle|\mathcal{O}\rangle \longmapsto \sum_{k,\ell=0}^{2^{n+1}-1} |k, \ell\rangle|[k]P + [\ell]Q\rangle. \tag{2}$$

After this step, apply Quantum Fourier Transform $\mathsf{QFT}_{2^{n+1}}$ to both exponent registers and then measure them. Finally, a classical post-processing procedure uses the measurement outcomes to reconstruct the discrete logarithm m with high probability, as shown in [Sho94].

In terms of resource, the dominant cost arises from the coherent group evaluation, i.e., the double-scalar multiplication on elliptic curve points. To reduce the qubit cost associated with QFT, its full circuit

[1] Hasse's bound [Has36] indicates that $\mathrm{ord}(P) \leq \#E(\mathbb{F}_p) \leq p + 1 + 2\sqrt{p}$, which can be represented by at most $n + 1$ bits.

on the exponent registers can be replaced by a semiclassical variant [GN96]. This semiclassical Fourier transform performs measurements during the computation, reusing qubits and applying conditional phase rotations based on previously observed outcomes. The overall quantum circuits of Shor's algorithm for ECDLP using QFT (resp. semiclassical QFT) are shown in Figure 2 (resp. Figure 3).

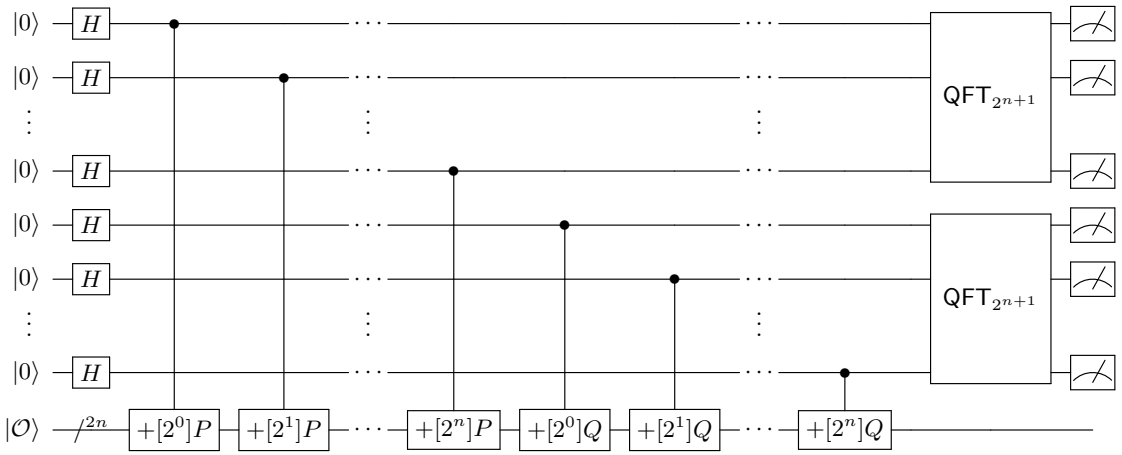


Figure 2: The overall quantum circuit of Shor's algorithm for solving ECDLP using QFT. The qubits, from top to bottom, correspond to the exponent registers containing k and ℓ in Equation 2 (ordered from lower-order bits to higher-order bits), and to the register that stores the elliptic curve point accumulator.

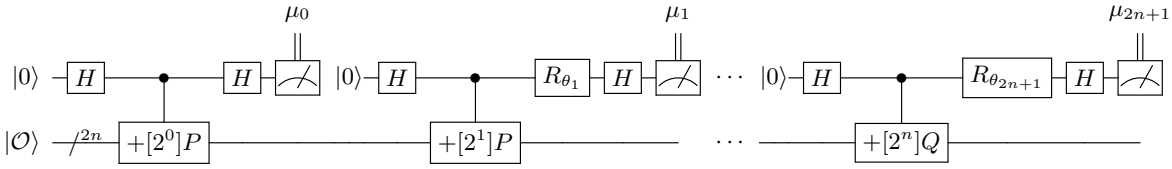


Figure 3: The overall quantum circuit of Shor's algorithm for solving ECDLP using semiclassical QFT. The gates R_{θ_i} denote rotation gates with rotation angle $\theta_i = \sum_{j=0}^{i-1} 2^{i-j}\mu_j$, where the values $\mu_j \in \{0, 1\}$ are outcomes obtained in previous measurements. By employing the semiclassical QFT, the exponent register requires only a single qubit, resulting in a reduction of $2n + 1$ qubits compared to the standard implementation.

3 Space-Efficient Extended Euclidean Algorithm

As discussed in previous sections, the dominant cost in designing quantum circuits for solving ECDLP via Shor's algorithm arises from coherent group additions on elliptic curve points. Among these operations, modular inversion is the most resource-consuming component. In this section, we present a space-efficient and reversible algorithm for modular inversion based on the Extended Euclidean Algorithm. This design is suitable for further translation into a quantum circuit, as detailed in Section 4.

Suppose that $p > x$ are two positive integers. The well-known *Euclidean algorithm* can be used to compute the greatest common divisor (GCD) of p and x. The algorithm initializes $r_0 := p$ and $r_1 := x$, and for iterations $i = 1, 2, \ldots$, it repeatedly divides r_{i-1} by r_i to obtain the quotient $q_i = \lfloor r_{i-1}/r_i \rfloor$ and

the remainder $r_{i+1} = r_{i-1} - q_i r_i$. The process terminates when $r_{k+1} = 0$, at which point the greatest common divisor of p and x equals r_k.

The *Extended Euclidean Algorithm (EEA)* not only computes the GCD of p and x, but also outputs the modular inverse $x^{-1} \bmod p$ when p and x are coprime. In addition to computing the sequence r_i, it also initializes $t_0 := 0$ and $t_1 := 1$, and for each iteration $i = 1, 2, \ldots$, updates $t_{i+1} = t_{i-1} + q_i t_i$ after obtaining $q_i = \lfloor r_{i-1}/r_i \rfloor$. When the algorithm first encounters $r_k = 1$, the modular inverse is given by $(-1)^{k-1} t_k \bmod p$.

When realizing the Extended Euclidean algorithm on a quantum computer to compute the modular inverse of x with a prime modulus p of n binary digits, the primary challenge lies in handling superpositions of x. The number of iterations required varies from 1 to $O(n)$ depending on the input, implying that a straightforward implementation would require $O(n)$ iterations. Each iteration involves divisions and multiplications on n-digit numbers, leading to an overall gate complexity of $O(n^3)$.

In what follows, we describe the dedicated four-phase algorithm framework first proposed in [PZ03], followed by our own algorithmic optimizations and the details of our optimized circuit implementation.

3.1 The four-phase algorithm framework

The four-phase algorithm framework was originally introduced in [PZ03]. It operates on six quantum registers, denoted as r_{i-1}, r_i, t_{i-1}, t_i, q_i, and ℓ, where the first five registers correspond to the variables used in the i-th iteration of EEA. To compute the quotient $q_i = \lfloor r_{i-1}/r_i \rfloor$, the algorithm performs a binary long-division procedure consisting solely of bit shifts and long additions or subtractions. The algorithm proceeds through the following four phases:

- **Phase 1.** The register containing r_i is repeatedly shifted left by one bit, while the shift counter ℓ is incremented at each step. This continues until the inequality $2^\ell r_i > r_{i-1}$ holds. Conceptually, this phase determines the largest power of two by which r_i can be multiplied without exceeding r_{i-1}. Equivalently, after the loop terminates, $\ell_0 := \ell - 1$ gives the highest bit position in the binary representation of q_i.

- **Phase 2.** The register containing r_i is then shifted right by one bit per step, with ℓ decremented accordingly. At each step, the registers corresponding to r_{i-1} and q_i are updated as

$$(r_{i-1} - q' r_i, q') \to \left(r_{i-1} - (q' + 2^\ell q_{i,\ell}) r_i, q' + 2^\ell q_{i,\ell} \right),$$

  depending on whether $r_{i-1} - q' r_i$ is at least $2^\ell r_i$. Here q' denotes the current partial quotient, given by $q' = 2^{\ell_0} q_{i,\ell_0} + 2^{\ell_0 - 1} q_{i,\ell_0 - 1} + \cdots + 2^{\ell+1} q_{i,\ell+1}$, representing the most significant bits accumulated so far in the binary expansion

$$q_i = 2^{\ell_0} q_{i,\ell_0} + 2^{\ell_0 - 1} q_{i,\ell_0 - 1} + \cdots + q_{i,0}.$$

  This iterative process ultimately yields the transformation

$$(r_{i-1}, 0) \to (r_{i-1} - q_i r_i, q_i),$$

  completing the computation of q_i.

- **Phase 3.** The register containing t_i is shifted left by one bit per step, and ℓ is incremented correspondingly. The registers associated with t_{i-1} and q_i are updated as

$$(t_{i-1} + q''t_i, q') \to \left(t_{i-1} + (q'' + 2^{\ell} q_{i,\ell})t_i, q'\right),$$

where the update depends on the bit value $q_{i,\ell}$ of q_i. After each step, $q_{i,\ell}$ is reset to 0, depending on whether $t_{i-1} + (q'' + 2^{\ell} q_{i,\ell})t_i > 2^{\ell} t_i$. Here q' and q'' represent the most and least significant portions of the quotient, respectively:

$$q' = 2^{\ell_0} q_{i,\ell_0} + 2^{\ell_0 - 1} q_{i,\ell_0 - 1} + \cdots + 2^{\ell} q_{i,\ell}, \quad q'' = 2^{\ell - 1} q_{i,\ell - 1} + 2^{\ell - 2} q_{i,\ell - 2} + \cdots + q_{i,0}.$$

After completing all steps, the registers corresponding to t_{i-1} and q_i are updated as

$$(t_{i-1}, q_i) \to (t_{i-1} + q_i t_i, 0).$$

- **Phase 4.** Finally, the register containing t_i is repeatedly shifted right by one bit, and ℓ is decremented at each step. The process terminates once $\ell = 0$.

Upon completion of all four phases, SWAP operations are applied between the register pairs corresponding to (r_{i-1}, r_i) (which is currently (r_{i+1}, r_i)) and the register pairs corresponding to (t_{i-1}, t_i) (which is currently (t_{i+1}, t_i)), respectively. This completes one full iteration of the EEA implementation.

We emphasize that, during the execution of this four-phase algorithm, the correspondence between the iteration index in EEA and the phase number may vary depending on the input value x. In other words, the algorithm's internal progression through iterations and phases is input-dependent. To avoid ambiguity in the subsequent descriptions, we use "step" to represent any sub-iteration occurring within any phase of an EEA iteration. The example in Table 3 illustrates how the step status evolves for different input values under this four-phase framework.

Although the sequence of step statuses differs for each input x, the total number of steps required to reach an index k such that $r_{k-1} = 1, r_k = 0$ can be bounded within the interval $[4n, 4\lceil cn \rceil]$ with $c = 3/\log_2(2 + \sqrt{3})$, as formally proven in Appendix A.1. This bounded-step property is crucial for quantum implementation: it implies that a quantum circuit corresponding to one "step" can be repeated at most $4\lceil cn \rceil < 6.316n$ times to definitely obtain the modular inverse of any input x in superposition. Since each step involves arithmetic operations such as shifts, additions, and subtractions, which require $O(n)$ quantum gates, the overall gate complexity of the algorithm is reduced to $O(n^2)$.

Keeping the above discussion in mind, the forward space-efficient EEA subroutine for computing the modular inverse $x^{-1} \bmod p$, where p is a prime represented with n binary digits and $x \in \{1, 2, \ldots, p-1\}$, is summarized in Algorithm 1. In this context: (i) the register allocation strategy is described in Section 3.2; (ii) Algorithm 3 provides the optimized stepwise iteration designed to minimize gate complexity and ensure quantum reversibility, as further discussed in Sections 3.2 and 3.3; (iii) when $x > p/2$, we run the EEA on $p - x$ instead of x and record this choice in the iteration-parity qubit.

3.2 Space-efficient algorithm by register sharing

In [PZ03], the authors introduced the concept of *register sharing*. The central idea of register sharing is that, during the execution of the EEA, the sequence $\{r_i\}$ is monotonically decreasing while the sequence

Input	x_1	x_2	x_3
Step 1	Iteration 1, Phase 1	Iteration 1, Phase 1	Iteration 1, Phase 1
Step 2	Iteration 1, Phase 1	Iteration 1, Phase 1	Iteration 1, Phase 1
Step 3	Iteration 1, Phase 2	Iteration 1, Phase 1	Iteration 1, Phase 1
Step 4	Iteration 1, Phase 2	Iteration 1, Phase 1	Iteration 1, Phase 2
Step 5	Iteration 1, Phase 3	Iteration 1, Phase 1	Iteration 1, Phase 2
Step 6	Iteration 1, Phase 3	Iteration 1, Phase 2	Iteration 1, Phase 2
Step 7	Iteration 1, Phase 4	Iteration 1, Phase 2	Iteration 1, Phase 3
Step 8	Iteration 1, Phase 4 (and SWAP)	Iteration 1, Phase 2	Iteration 1, Phase 3
Step 9	Iteration 2, Phase 1	Iteration 1, Phase 2	Iteration 1, Phase 3
Step 10	Iteration 2, Phase 1	Iteration 1, Phase 2	Iteration 1, Phase 4
Step 11	Iteration 2, Phase 1	Iteration 1, Phase 3	Iteration 1, Phase 4
Step 12	Iteration 2, Phase 2	Iteration 1, Phase 3	Iteration 1, Phase 4 (and SWAP)
Step 13	Iteration 2, Phase 2	Iteration 1, Phase 3	Iteration 2, Phase 1
Step 14	Iteration 2, Phase 2	Iteration 1, Phase 3	Iteration 2, Phase 1
Step 15	Iteration 2, Phase 3	Iteration 1, Phase 3	Iteration 2, Phase 2
Step 16	Iteration 2, Phase 3	Iteration 1, Phase 4	Iteration 2, Phase 2
...	...	...	...
Step 100	Iteration 9, Phase 3	Iteration 5, Phase 2	Iteration 8, Phase 4
...	...	...	...

Table 3: An example illustrating that the EEA iteration index and phase number may vary depending on the input value x.

Algorithm 1 Forward space-efficient EEA inversion subroutine

Require: Quantum registers $|x\rangle|0^n\rangle|0^m\rangle$, where the first n-qubit register is the input/output register, the second n-qubit register is a large workspace, and the remaining m qubits serve as auxiliary qubits.

Ensure: Output state $|x^{-1} \bmod p\rangle|0^n\rangle|\Gamma(x)\rangle$, where $|\Gamma(x)\rangle$ denotes the remaining forward EEA state.

Initialize register `Work1` with $(n+3)$ qubits as $|100, p\rangle$ ▷ Using $(n+3)$ auxiliary qubits

Initialize register `Work2` with $(n+3)$ qubits as $|000, x\rangle$ ▷ Pad x to n bits on the left

Initialize `Length` registers to $|\ell_t := 1, \ell_q := 0, \ell_{r'} :=$ length of $x, \ell_s := 0\rangle$

Initialize `Control` registers to $|$`Phase1` $:= 0,$ `Phase2` $:= 0,$ `Sign` $:= 0,$ `Iter` $:= 0\rangle$

if $x > p/2$ **then** ▷ Ensures that the algorithm is invertible

  `Iter` $\leftarrow$ `Iter` $\oplus 1$

end if

if `Iter` $= 1$ **then**

  $x \leftarrow p - x$

end if

for $i = 1, 2, \cdots, 4\lceil cn \rceil$ **do** ▷ Main iterative loop performing the optimized algorithm steps

  Execute Algorithm 3 on the quantum registers

end for

Keep the t' part of `Work2` as the output value

if `Iter` $= 0$ **then** ▷ Convert t' to the positive inverse modulo p

  $t' \leftarrow p - t'$

end if

Reset the input-independent terminal contents of the large workspace to $|0^n\rangle$

Denote the remaining forward EEA state by $|\Gamma(x)\rangle$

$\{t_i\}$ is monotonically increasing. This complementary behavior allows both values to occupy the same quantum register at different stages of computation, thereby reducing the overall space requirement. The identity

$$r_{i-1}t_i + r_i t_{i-1} = p$$

implies that two quantum registers, each consisting of $(n+2)$ qubits, are sufficient: one to store the pair (r_{i-1}, t_i) and the other to store (r_i, t_{i-1}).

We extend this register-sharing idea further. Specifically, we observe that a single quantum register of $(n+2)$ qubits can be used to store the triple (r_{i-1}, t_i, q_i) simultaneously, including the intermediate states that arise during phases 2 and 3 of the algorithm. These phases are responsible for computing $q_i = \lfloor r_{i-1}/r_i \rfloor$ and updating the variables

$$r_{i-1} \leftarrow r_{i-1} - q_i r_i, t_{i-1} \leftarrow t_{i-1} + q_i t_i.$$

The detailed allocation of quantum registers is described as follows, and illustrated in Figure 4.

- `Work1` **register (of** $(n+3)$ **qubits).** The left-most $\ell_t + 1$ qubits encode the value $t := t_i$ in little-endian order (here we append a zero to the rightmost position of t in order to simplify the circuit implementation). The following ℓ_q qubits store the intermediate quotient value $q := q'$ (only the effective most significant bits) in phases 2 and 3, in big-endian order. The remaining qubits hold the value $r := r_{i-1}$ (or its updated form $r := r_{i-1} - q'r_i$) in big-endian order.

- `Work2` **register (of** $(n+3)$ **qubits).** The right-most $\ell_{r'}$ qubits store $r' := r_i$ in big-endian order, while the remaining qubits store the value $t' := t_{i-1}$ (and its intermediate update $t' := t_{i-1} + q''t_i$) in little-endian order. During phases 2 and 3, this register may be circularly shifted left by ℓ_s positions. In such cases, the value t' may span both ends of the register, effectively splitting into two contiguous parts.

- `Length` **registers.** Four auxiliary registers are used to manage variable-length storage. Each of them contains $\lfloor \log_2 n \rfloor + 1$ qubits. They store the length indicators $\ell_t, \ell_q, \ell_{r'}$, and the shift counter ℓ_s respectively. These `Length` registers mark the logical boundaries within the `Work1` and `Work2` registers and enable coherent manipulation of superposed inputs of varying lengths.

- `Control` **registers.** Two single-qubit `Phase` registers indicate the current phase of the four-phase algorithmic framework, and one single-qubit `Iter` register that records the parity of the current EEA iteration number. Additionally, there are two auxiliary single-qubit registers, `Sign` and `Ctrl`, used for intermediate flag and control operations.

We next describe the behavior of the system when performing comparison, addition, and subtraction operations on the register pairs (r, r') and (t, t'). The arithmetic between r and r' is relatively straightforward: a left shift by ℓ_s positions corresponds to multiplying r' by 2^{ℓ_s}. In contrast, the arithmetic involving t and t' requires a more careful interpretation. A convenient way to view this operation is that a left shift of ℓ_s positions can be regarded as dividing t' by 2^{ℓ_s}, where the integer part is placed in the left-most portion of `Work2`, and the fractional part occupies the right-most portion. When t is added to the integer part of t' (which is properly aligned), the arithmetic relation can be expressed as

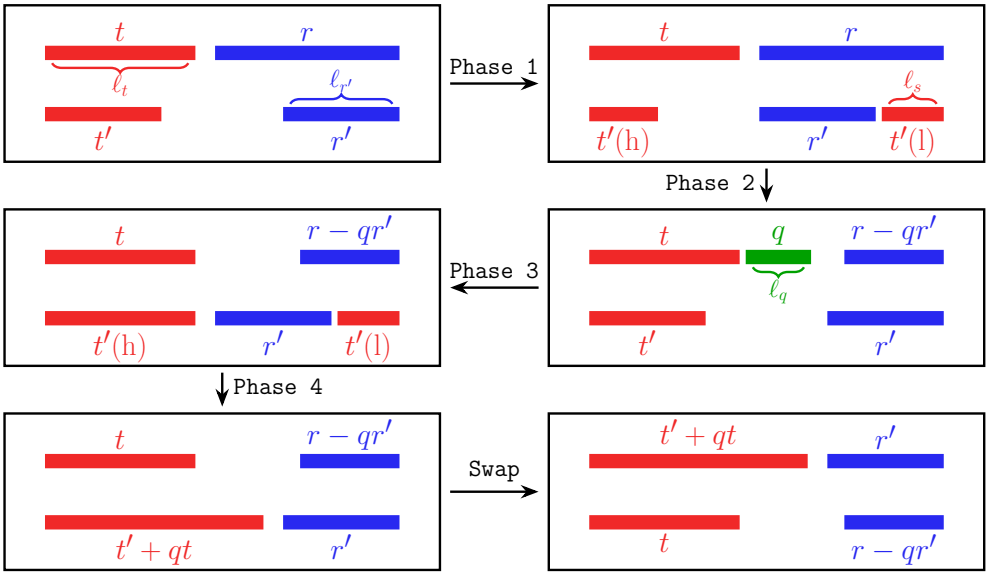


Figure 4: An illustration of how the two `Work` registers are allocated for temporary variables within a single iteration of the EEA. The upper and lower stripes represent the `Work1` and `Work2` registers, respectively. The symbols (h) and (l) indicate that the value t' is split into its higher-order bits (h) and lower-order bits (l), which are placed at the corresponding positions on the two sides of the `Work2` register.

$2^{-\ell_s}t' + t = 2^{-\ell_s}(t' + 2^{\ell_s}t)$. Thus, the expression $t' + 2^{\ell_s}t$ represents the desired result, corresponding to a left shift by ℓ_s positions.

With this register allocation, one step of our space-efficient EEA proceeds as in Algorithm 2. To illustrate how the proposed space-efficient EEA operates in practice, we also provide a concrete execution example for $p = 37$ and $x = 13$, shown in Table 4.

For each step, Table 4 shows how the contents of the two `Work` registers evolve and how different variables are mapped onto `Work1` and `Work2` under the register-sharing strategy. In particular, the variable t' stored in `Work2` is divided into two parts that occupy the left-most and right-most portions of the register. The division symbols are not explicitly implemented in the quantum circuit; they are all illustrative and inferred from the corresponding `Length` registers, used to visualize the register allocation. In the actual quantum implementation, all registers are in superposition, and the location of these division symbols may vary across different input values.

3.3 Algorithmic optimizations

The baseline implementation of one step, as described in the previous subsection, was constructed to be fully reversible, ensuring that every state transformation can be realized as a unitary operation on a quantum computer. However, this direct implementation contains multiple redundant operations that unnecessarily increase both the circuit depth and gate count. In this subsection, we present a series of algorithmic optimizations applied to the baseline design in order to reduce the quantum circuit depth

Algorithm 2 One step of our EEA implementation with register sharing

Require: `Work1` register stores $|t, q, r\rangle$, `Work2` register stores $|t', r'\rangle$
Require: `Length` registers store $|\ell_t, \ell_q, \ell_{r'}, \ell_s\rangle$
Require: `Control` registers store $|\texttt{Phase1}, \texttt{Phase2}, \texttt{Iter}, \texttt{Sign}\rangle$.

if $(\texttt{Phase1}, \texttt{Phase2}) = (0, 0)$ **then** ▷ Arithmetic logic for the four phases
  Perform a one-position left shift on `Work2`
  $\ell_s \leftarrow \ell_s + 1$
  $\texttt{Sign} \leftarrow \texttt{Sign} \oplus (r < 2^{\ell_s} r')$ ▷ Act on qubits $(\ell_t + \ell_q + 2)$ through $(n + 3 - \ell_s)$
else if $(\texttt{Phase1}, \texttt{Phase2}) = (0, 1)$ **then**
  Perform a one-position right shift on `Work2`
  $\ell_s \leftarrow \ell_s - 1$, $\ell_q \leftarrow \ell_q + 1$
  $(\texttt{Sign}, r) \leftarrow (\texttt{Sign}, r) - 2^{\ell_s} r'$ ▷ Use register `Sign` to store the sign of the subtraction result
  if $\texttt{Sign} = 1$ **then**
    $r \leftarrow r + 2^{\ell_s} r'$ ▷ Ignore overflows
  end if
  $\texttt{Sign} \leftarrow \texttt{Sign} \oplus 1$
  Swap `Sign` with the $(\ell_t + \ell_q + 1)$-th qubit of `Work1` ▷ The least significant work qubit of q
else if $(\texttt{Phase1}, \texttt{Phase2}) = (1, 0)$ **then**
  Swap `Sign` with the $(\ell_t + \ell_q + 1)$-th qubit of `Work1`
  if $\texttt{Sign} = 0$ **then**
    $t' \leftarrow t' - 2^{\ell_s} t$ ▷ Act on left-most $(\ell_t + 1)$ qubits
  end if
  $\texttt{Sign} \leftarrow \texttt{Sign} \oplus 1$
  $(\texttt{Sign}, t') \leftarrow (\texttt{Sign}, t') + 2^{\ell_s} t$ ▷ Use register `Sign` to store the sign of the addition result
  Perform a one-position left shift on `Work2`
  $\ell_s \leftarrow \ell_s + 1$, $\ell_q \leftarrow \ell_q - 1$
else if $(\texttt{Phase1}, \texttt{Phase2}) = (1, 1)$ **then**
  $\texttt{Sign} \leftarrow \texttt{Sign} \oplus (t' \geq 2^{\ell_s} t)$ ▷ Same qubit range as above
  Perform a one-position right shift on `Work2`
  $\ell_s \leftarrow \ell_s - 1$
end if
if $\ell_q = 0$ **and** $\ell_{r'} > 0$ **then** ▷ Phase update logic; $\ell_{r'} = 0$ indicates algorithm termination
  $\texttt{Phase2} \leftarrow \texttt{Phase2} \oplus \texttt{Sign} \oplus \texttt{Phase1}$, $\texttt{Sign} \leftarrow \texttt{Sign} \oplus \texttt{Phase2}$
end if
if $\ell_s = 0$ **then**
  $\texttt{Phase1} \leftarrow \texttt{Phase1} \oplus 1$, $\texttt{Phase2} \leftarrow \texttt{Phase2} \oplus 1$
end if
if $\ell_q = 0$ **and** $\ell_s = 0$ **then** ▷ Register swapping at the end of one EEA iteration
  Swap `Work1` and `Work2`
  Update ℓ_t to the bit length of the new t ▷ Act on the left-most $(n + 3 - \ell_{r'})$ qubits of `Work1` and `Work2`
  Update $\ell_{r'}$ to the bit length of the new r' ▷ Act on the right-most $(n + 2 - \ell_t)$ qubits of `Work1` and `Work2`
  $\texttt{Iter} \leftarrow \texttt{Iter} \oplus 1$
end if

Step	Work1	Work2	t	q	r	t'	r'	ℓ_t	ℓ_q	$\ell_{r'}$	ℓ_s	Phase1	Phase2	Iter	Sign
0	10||0100101	00000|1101|	1	0	37	0	13	1	0	4	0	0	0	0	0
1	10||0100101	0000|1101|0	1	0	37	0	13	1	0	4	1	0	0	0	0
2	10||0100101	000|1101|00	1	0	37	0	13	1	0	4	2	0	1	0	0
3	10|1|001011	0000|1101|0	1	2	11	0	13	1	1	4	1	0	1	0	0
4	10|10|01011	00000|1101|	1	2	11	0	13	1	2	4	0	1	0	0	0
5	10|1|001011	0000|1101|0	1	2	11	0	13	1	1	4	1	1	0	0	0
6	10||0001011	000|1101|01	1	0	11	2	13	1	0	4	2	1	1	0	1
7	10||0001011	1000|1101|0	1	0	11	2	13	1	0	4	1	1	1	0	0
8	010||001101	10000|1011|	2	0	13	1	11	2	0	4	0	0	0	1	0
9	010||001101	0000|1011|1	2	0	13	1	11	2	0	4	1	0	1	1	0
10	010|1|00010	10000|1011|	2	1	2	1	11	2	1	4	0	1	0	1	0
11	010||000010	1000|1011|1	2	0	2	3	11	2	0	4	1	1	1	1	1
12	110||001011	0100000|10|	3	0	11	2	2	2	0	2	0	0	0	0	0
13	110||001011	100000|10|0	3	0	11	2	2	2	0	2	1	0	0	0	0
14	110||001011	00000|10|01	3	0	11	2	2	2	0	2	2	0	0	0	0
15	110||001011	0000|10|010	3	0	11	2	2	2	0	2	3	0	1	0	0
16	110|1|00011	00000|10|01	3	4	3	2	2	2	1	2	2	0	1	0	0
17	110|10|0011	100000|10|0	3	4	3	2	2	2	2	2	1	0	1	0	0
18	110|101|001	0100000|10|	3	5	1	2	2	2	3	2	0	1	0	0	0
19	110|10|0001	010000|10|1	3	4	1	5	2	2	2	2	1	1	0	0	0
20	110|1|00001	10000|10|10	3	4	1	5	2	2	1	2	2	1	0	0	0
21	110||000001	0100|10|100	3	0	1	17	2	2	0	2	3	1	1	0	1
22	110||000001	00100|10|10	3	0	1	17	2	2	0	2	2	1	1	0	0
23	110||000001	000100|10|1	3	0	1	17	2	2	0	2	1	1	1	0	0
24	100010||010	11000000|1|	17	0	2	3	1	5	0	1	0	0	0	1	0
25	100010||010	1000000|1|1	17	0	2	3	1	5	0	1	1	0	0	1	0
26	100010||010	000000|1|11	17	0	2	3	1	5	0	1	2	0	1	1	0
27	100010|1|00	1000000|1|1	17	2	0	3	1	5	1	1	1	0	1	1	0
28	100010|10|0	11000000|1|	17	2	0	3	1	5	2	1	0	1	0	1	0
29	100010|1|00	1000000|1|1	17	2	0	3	1	5	1	1	1	1	0	1	0
30	100010||000	100100|1|10	17	0	0	37	1	5	0	1	2	1	1	1	1
31	100010||000	0100100|1|1	17	0	0	37	1	5	0	1	1	1	1	1	0
32	Terminated	Terminated	37	0	1	17	0	6	0	0	0	0	0	0	0
33	Terminated	Terminated	37	0	1	17	0	6	0	0	1	0	0	0	0
34	Terminated	Terminated	37	0	1	17	0	6	0	0	2	0	0	0	0
35	Terminated	Terminated	37	0	1	17	0	6	0	0	3	0	0	0	0
36	Terminated	Terminated	37	0	1	17	0	6	0	0	4	0	0	0	0

Table 4: An execution example of our space-efficient EEA for $p = 37$ and $x = 13$. The table mainly illustrates how the two $(n + 3)$-qubit registers `Work1` and `Work2` are allocated during the execution to store the variables t, q, r and t', r', respectively.

and overall gate complexity.

Removing redundant arithmetic and shift operations. In the baseline version, several arithmetic and shift operations between different phases are functionally redundant. Specifically, we have:

In phases 1 and 2:	comparison, addition, subtraction between $(r, 2^{\ell_s} r')$;
In phases 3 and 4:	comparison, addition, subtraction between $(t', 2^{\ell_s} t)$;
In phases 1 and 3:	a one-position left shift on `Work2`;
In phases 2 and 4:	a one-position right shift on `Work2`.

By combining the control conditions of these operations (or equivalently, the corresponding quantum control wires), we can eliminate redundant operations from the circuit. This optimization results in a total of two groups of location-controlled addition/subtraction operations and two location-controlled swaps in one step. Here, *location-control* refers to arithmetic operations applied conditionally on specific positions of the two `Work` registers, with the active positions indicated by the `Length` registers.

Merging location-controlled swaps. To further decrease the circuit depth and gate count, we observe that the two location-controlled swaps appearing in phases 2 and 3 act on the same qubit position of `Work1`. We merge them into a single swap controlled by `Phase1` $\oplus$ `Phase2`. The required update of ℓ_q depends on the phase: ℓ_q is increased in phase 2 and decreased in phase 3.

Algorithm pseudocode and overall complexity. After applying the above optimizations, the overall asymptotic quantum resource estimation per step is summarized as follows:

- Five location-controlled arithmetic operations, including one swap, two additions, and two subtractions; along with four length updates at the termination step of an EEA iteration. Each of these operations costs $O(n)$ gates in the implementation presented in Section 4.
- Four position shifts and one $(n+3)$-qubit SWAP, each requiring $O(n)$ quantum gates.

With these optimizations, one step of our space-efficient EEA proceeds as in Algorithm 3.

4 Quantum Circuit Implementation of Modular Inversion

In this section, we present the explicit quantum circuit implementation of a single step of the space-efficient EEA with register sharing, as described in Algorithm 3. Throughout the section, we use the same quantum register notation as in the algorithm.

The full circuit construction is, however, too complex to be depicted within a small number of circuit diagrams. This difficulty is mainly due to the presence of location-controlled arithmetic operations, which are highly non-standard. To present the construction in a relatively more explicit manner, we first describe the overall circuit structure in Section 4.1, where the location-controlled arithmetic operations

Algorithm 3 Optimized one step of our EEA implementation

Require: `Work1` register stores $|t, q, r\rangle$, `Work2` register stores $|t', r'\rangle$;
Require: `Length` registers store $|\ell_t, \ell_q, \ell_{r'}, \ell_s\rangle$;
Require: `Control` registers store $|$`Phase1`, `Phase2`, `Iter`, `Sign`$\rangle$.

if `Phase1` $= 0$ **then** ▷ Pre-shift operations
  Perform a one-position left shift on `Work2`, and update $\ell_s \leftarrow \ell_s + 1$
  if `Phase2` $= 1$ **then**
    Perform a two-position right shift on `Work2`, and update $\ell_s \leftarrow \ell_s - 2$
  end if
  (`Sign`, r) $\leftarrow$ (`Sign`, r) $- 2^{\ell_s} r'$ ▷ Arithmetic block 1: location-controlled subtraction on r's
  if `Phase2` $= 1$ **then**
    `Sign` $\leftarrow$ `Sign` $\oplus 1$
  end if
  if `Phase2` $= 0$ **or** `Sign` $= 0$ **then**
    $r \leftarrow r + 2^{\ell_s} r'$
  end if
end if
if `Phase1` $\oplus$ `Phase2` $= 1$ **then** ▷ Arithmetic block 2: location-controlled swap
  Swap `Sign` with the $(\ell_t + \ell_q + 1)$-th qubit of `Work1`
  if `Phase1` $= 1$ **then**
    $\ell_q \leftarrow \ell_q - 1$
  else
    $\ell_q \leftarrow \ell_q + 1$
  end if
end if
if `Phase1` $= 1$ **then** ▷ Arithmetic block 3: location-controlled addition on t's
  if `Phase2` $= 1$ **or** `Sign` $= 0$ **then**
    $t' \leftarrow t' - 2^{\ell_s} t$
  end if
  `Sign` $\leftarrow$ `Sign` $\oplus 1$
  (`Sign`, t') $\leftarrow$ (`Sign`, t') $+ 2^{\ell_s} t$
  Perform a one-position left shift on `Work2`, and update $\ell_s \leftarrow \ell_s + 1$ ▷ Post-shift operations
  if `Phase2` $= 1$ **then**
    Perform a two-position right shift on `Work2`, and update $\ell_s \leftarrow \ell_s - 2$
  end if
end if
if $\ell_q = 0$ **and** $\ell_{r'} > 0$ **then** ▷ Phase update logic; $\ell_{r'} = 0$ indicates algorithm termination
  `Phase2` $\leftarrow$ `Phase2` $\oplus$ `Sign` $\oplus$ `Phase1`, `Sign` $\leftarrow$ `Sign` $\oplus$ `Phase2`
end if
if $\ell_s = 0$ **then**
  `Phase1` $\leftarrow$ `Phase1` $\oplus 1$, `Phase2` $\leftarrow$ `Phase2` $\oplus 1$
end if
if $\ell_q = 0$ **and** $\ell_s = 0$ **then** ▷ Register swapping at the end of one EEA iteration
  Swap `Work1` and `Work2`
  Update ℓ_t to the bit length of the new t, $\ell_{r'}$ to the bit length of the new r'
  `Iter` $\leftarrow$ `Iter` $\oplus 1$
end if

are treated as circuit blocks; we then provide a detailed implementation of these operations in Section 4.4. In all circuit diagrams, the black triangle denotes the output wire that carries the result of the corresponding circuit block; and we denote $\ell = \lfloor \log_2 n \rfloor$ for convenience.

4.1 Overall circuit implementation of one step

According to Algorithm 3, the overall circuit implementation of a single iteration of our space-efficient EEA is shown in Figures 5 and 6, presented in a continuous layout. Both circuit diagrams are divided into several algorithmic components, indicated by dashed boxes. Each component corresponds exactly to one arithmetic block specified in Algorithm 3.

In both circuit diagrams, Shift denotes a position shift of the workspace: a positive value represents a left shift, while a negative value represents a right shift. The circuit blocks Add, Sub, and Swap denote location-controlled addition, subtraction, and swap operations, respectively. At the end of each EEA iteration, the circuit block labeled SWAP represents a full SWAP operation applied to all qubits in the registers `Work1` and `Work2`; this block is distinct from the Swap block to avoid ambiguity. Finally, the Len block denotes the update operations on the length registers.

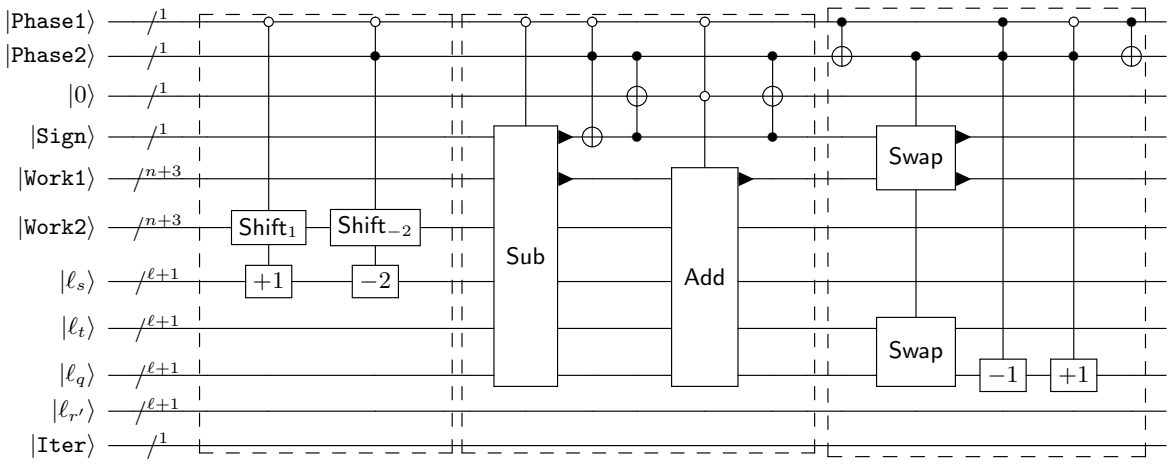


Figure 5: Overall circuit implementation (first half) of a single iteration of our space-efficient EEA. The three dashed boxes, from left to right, represent: (1) pre-shift operations; (2) location-controlled subtraction on r's; (3) location-controlled swap.

4.2 Step-dependent active windows

To reduce the quantum gate complexity for implementing our space-efficient EEA, we introduce step-dependent active windows for each location-controlled operation. As illustrated in the figures of the previous subsection, we define k and K as known bounds on the active indices of the `Work` registers.

For instance, at step T, a location-controlled Swap circuit acts on the $(\ell_t + \ell_q + 1)$-th qubit of the `Work1` register. Over all possible inputs x used to compute the modular inverse x^{-1} mod p, the quantity $\ell_t+\ell_q+1$ is bounded below by $k := k(T)$ and above by $K := K(T)$. Therefore, applying location-controlled

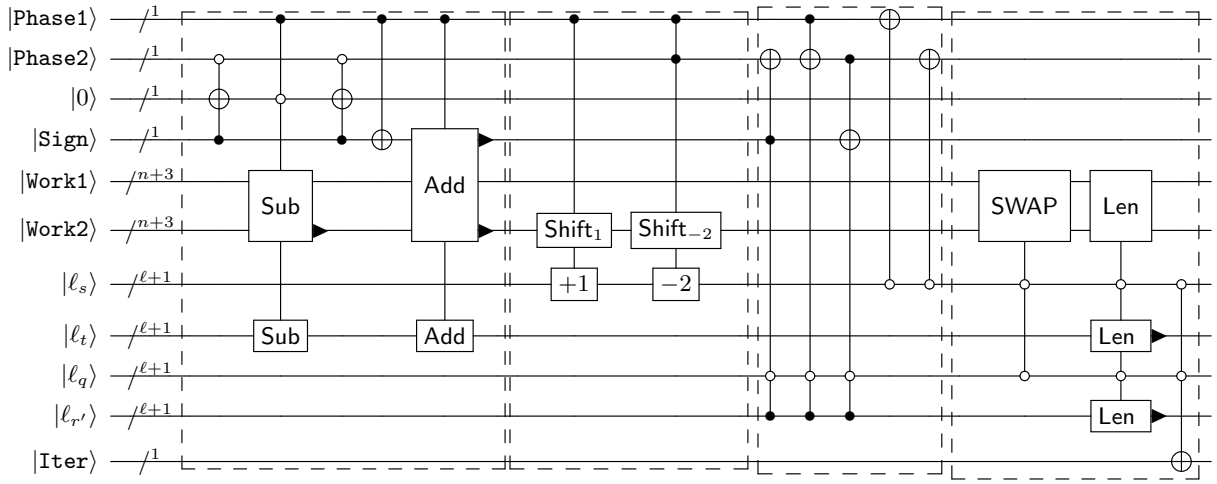


Figure 6: Overall circuit implementation (second half) of a single iteration of our space-efficient EEA, continued from Figure 5. The four dashed boxes, from left to right, represent: (1) location-controlled addition on t's; (2) post-shift operations; (3) phase update; (4) swapping `Work1` and `Work2`, together with the corresponding length updates at the end of one EEA iteration.

operations to qubits in the `Work1` register with indices outside the interval $[k, K]$ is unnecessary. We refer to this interval as the *active window*.

We next specify the step-dependent active windows for each type of location-controlled operation. All bounds are taken over all *possible inputs* x, namely, those inputs x for which the location-controlled circuits are activated by the outer control qubits. We defer the proofs to Appendix A.2. Here the constants are set to be

$$\lambda := (2+\sqrt{3})^{1/3}, \quad c := 1/\log_2 \lambda = 3/\log_2(2+\sqrt{3}), \quad \delta := \log_\lambda \left(\frac{4\sqrt{3}-3}{13} \lambda^2 \right) = 0.726208\ldots.$$

- **Location-controlled addition/subtraction on r's.** These operations act on qubits indexed from $(\ell_t + \ell_q + 2)$ to $(n + 3 - \ell_s)$. The upper bound $K_1 = K_1(T) = n + 3$; the lower bound $k_1 = k_1(T)$ is defined by

$$\ell_t + \ell_q + 2 \geq k_1(T) := \max\left\{ \left\lceil \frac{T - n - 1 - 4\delta}{4c - 1} \right\rceil, 1 \right\} + 2.$$

- **Location-controlled swap.** These operations act on the $(\ell_t + \ell_q + 1)$-th qubit. The two-sided bounds are defined by

$$\ell_t + \ell_q + 1 \geq k_2(T) := \max\left\{ \left\lceil \frac{T - 3(n+1) - 4\delta}{4c - 3} \right\rceil, 1 \right\} + 1,$$
$$\ell_t + \ell_q + 1 \leq K_2(T) := \min\left\{ \lfloor T/2 \rfloor + 2, n + 2 \right\}.$$

- **Location-controlled addition/subtraction on t's.** These operations act on the first $(\ell_t + 1)$ qubits. This is a one-sided controlled operation, the upper bound $K_3(T)$ is defined by

$$\ell_t + 1 \leq K_3(T) := \min\left\{ \lceil T/4 \rceil + 1, n + 1 \right\}.$$

- **Length-update circuit for updating ℓ_t.** We execute the length-update circuit only at step indices T that are multiples of four. This circuit acts on qubits indexed from ℓ_t to $(n+3-\ell_{r'})$, and the corresponding two-sided active window is defined by

$$\ell_t \geq k_4(T) := \max\left\{\left\lceil \frac{T-4(n+1)-4\delta}{4c-4}\right\rceil, 1\right\},$$
$$n+3-\ell_{r'} \leq K_4(T) := \min\left\{T/4+3, n+3\right\}.$$

- **Length-update circuit for updating $\ell_{r'}$.** Let $\ell_t^*, \ell_{r'}^*$ denote the updated value of $\ell_t, \ell_{r'}$ after the length update. This circuit acts on qubits indexed from ℓ_t^*+2 to $(n+4-\ell_{r'}^*)$, and the corresponding two-sided active window is defined by

$$\ell_t^* + 2 \geq k_5(T) := \max\left\{\left\lceil \frac{T-4\delta}{4c}\right\rceil, 1\right\},$$
$$n+4-\ell_{r'}^* \leq K_5(T) := \min\left\{T/4+4, n+3\right\}.$$

4.3 Unary iteration

We will use unary iteration [BBG+18] to implement location-controlled operations on a known interval of the two `Work` registers. Let L be a `Length` register promised to store a value $a \in \{k, k+1, \ldots, K\}$, let W be a `Work` register, and let c be an optional external control qubit. The operation has the form

$$\mathcal{U}_{[k,K]} = |0\rangle\langle 0|_c \otimes I + |1\rangle\langle 1|_c \otimes \sum_{j=k}^{K} |j\rangle\langle j|_{\mathsf{L}} \otimes U_j,$$

where U_j is a unitary operation on W selected by the value j. Equivalently, the operation performs $|c\rangle|a\rangle|\psi\rangle \mapsto |c\rangle|a\rangle U_a^c|\psi\rangle$ for all $a \in [k, K]$.

The main primitive used by unary iteration is a reversible AND. The compute and uncompute forms of this gate are shown in Figure 7. In the Toffoli/CNOT counting convention used here, computing an AND costs one Toffoli gate, while uncomputing an AND using measurement costs one CNOT gate with no Toffoli gates.

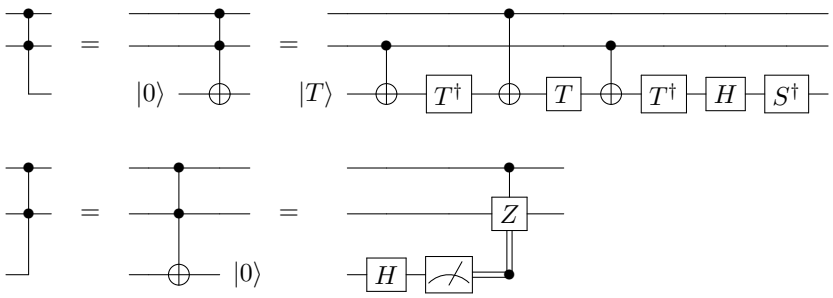


Figure 7: Compute and uncompute forms of the reversible AND primitive, reproduced from [BBG+18].

For a general interval, unary iteration can be viewed as a binary tree whose leaves are the labels $k, k+1, \ldots, K$. Each internal node stores a subset of labels. If the current node has control bit g, we

choose the most significant bit position b that is not constant on its labels, thereby separating them into two nonempty subsets according to the value of A_b. For an increasing-order traversal, we first compute $h = g(1 - A_b)$ with one reversible AND, use h as the control for the $A_b = 0$ child, then switch the same temporary bit by a CNOT, $h \leftarrow h \oplus g$, so that $h = gA_b$ controls the $A_b = 1$ child. After the second child is processed, h is uncomputed. At a leaf j, the current control bit is exactly

$$e_j = c[a = j],$$

and the circuit applies U_j controlled on e_j. For a decreasing-order traversal, the two children are visited in the opposite order. Figure 8 illustrates this construction for the interval $\{5, 6, 7, 8\}$.

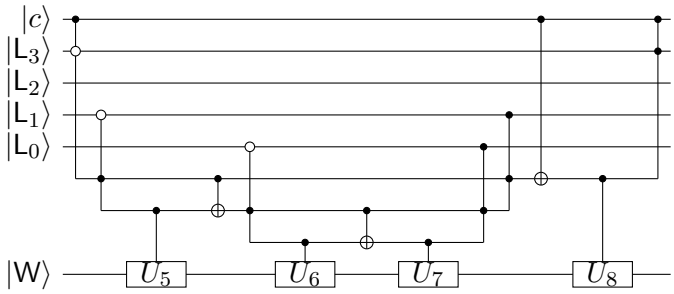


Figure 8: Pruned unary iteration on the promised interval $A \in \{5, 6, 7, 8\}$. The labels are split by the bits of A only when the split separates the remaining candidates. The temporary control is reused to visit the two children of each split, and the leaves apply U_5, U_6, U_7, U_8 in increasing order.

If $M = K - k + 1$, the tree has M leaves and $M - 1$ internal nodes. Thus the selection logic contributes $M - 1 = K - k$ Toffoli gates, while the leaf operations contribute the cost of the controlled U_j gates themselves. For $M \geq 2$, the recursion depth, and hence the number of temporary control qubits, is at most $\lceil \log_2(M - 1) \rceil + 1$. For the trivial case $M = 1$, no temporary control qubit is needed. This replaces independent equality tests for all j, which would cost $O(M \log_2 n)$ Toffoli gates for an $O(\log_2 n)$-bit index register, by a linear-size selection circuit using only $O(\log_2 M)$ temporary control qubits.

4.4 Location-controlled operations

In this subsection, the two `Work` registers are indexed, from left to right, by $u_1, u_2, \ldots, u_{n+3}$ and $v_1, v_2, \ldots, v_{n+3}$, respectively.

Location-controlled Swap circuit. We first consider the location-controlled swap operation appearing in Figure 5. The goal is to swap the `Sign` qubit with the $(\ell_t + \ell_q + 1)$-th qubit of `Work1`, while the whole operation is controlled by an external qubit `Ctrl`. Let $J = \ell_t + \ell_q + 1$, where k and K are known constants such that $k \leq J \leq K$. After temporarily storing J in the ℓ_q register, the location-controlled swap can be written as

$$\mathsf{Swap}_{[k,K]} = |0\rangle\langle 0|_{\texttt{Ctrl}} \otimes I + |1\rangle\langle 1|_{\texttt{Ctrl}} \otimes \sum_{j=k}^{K} |j\rangle\langle j|_{\ell_q} \otimes \mathsf{SWAP}(\texttt{Sign}, u_j)\,,$$

where the subscript ℓ_q denotes the temporary register storing J.

This operation can be implemented directly using unary iteration. We first reversibly transform the ℓ_q register as $|\ell_t\rangle|\ell_q\rangle \longmapsto |\ell_t\rangle|\ell_t + \ell_q + 1\rangle$. Using `Ctrl` as the global control, pruned unary iteration over the labels $k, k+1, \ldots, K$ then produces, one value at a time, the control bit $e_j = \texttt{Ctrl} \wedge [J = j]$. When this unary control is active, we swap `Sign` and u_j. After it finishes we restore the ℓ_q register to its original value. Figure 9 illustrates this construction for the example $k = 5$ and $K = 8$.

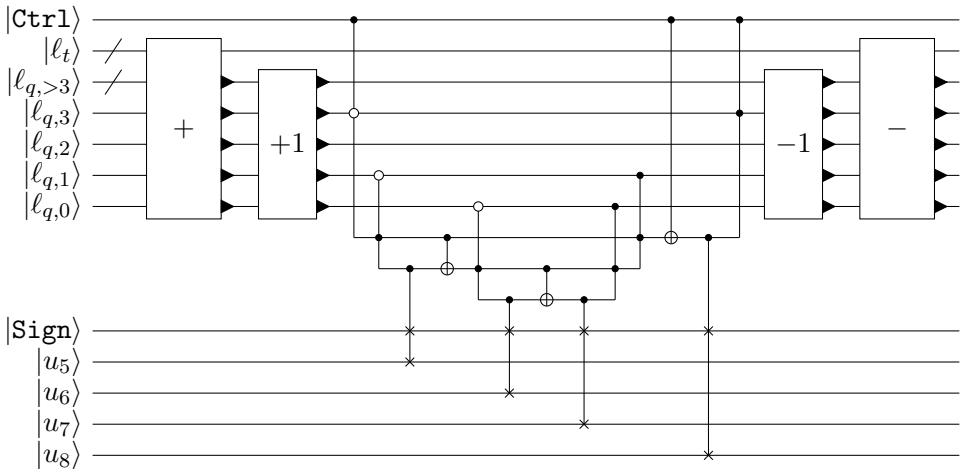


Figure 9: Location-controlled swap circuit for $k = 5$ and $K = 8$. After computing $J = \ell_t + \ell_q + 1$, pruned unary iteration over the labels $5, 6, 7, 8$ selects the work qubit to be swapped with `Sign`.

The unary iterator contributes $K - k + O(1)$ Toffoli gates and $2(K - k) + O(1)$ CNOT gates, the controlled swaps contribute $K - k + O(1)$ Toffoli gates and $2(K - k) + O(1)$ CNOT gates. The reversible transformations of the ℓ_q register contribute $O(\log_2 n)$ Toffoli/CNOT gates. Thus the location-controlled Swap circuit uses $2(K - k) + O(\log_2 n)$ Toffoli gates and $4(K - k) + O(\log_2 n)$ CNOT gates.

Location-controlled Add, Sub circuit. We next describe the location-controlled addition and subtraction circuits. As a representative example, consider the operation

$$(\texttt{Sign}, r) \leftarrow (\texttt{Sign}, r) - 2^{\ell_s} r', \tag{3}$$

where `Sign` records the sign of the subtraction result. In terms of physical positions in the two `Work` registers, this operation acts only on the interval

$$L = \ell_t + \ell_q + 2, \qquad\qquad R = n + 3 - \ell_s.$$

That is, the relevant qubits are $u_L, \ldots, u_R$ in `Work1` and $v_L, \ldots, v_R$ in `Work2`; outside this interval all qubits are left unchanged. The shift by ℓ_s is already reflected in the endpoint $R = n + 3 - \ell_s$, so the aligned bits $v_L, \ldots, v_R$ represent the contribution of $2^{\ell_s} r'$ to the subtraction. For a known active window $[k, K]$, we promise that $k \le L \le R \le K$.

If the endpoints L and R were fixed, the subtraction could be implemented by a ripple-carry subtractor on the qubit pairs (u_j, v_j) for $j \in [L, R]$ with a dedicated carry qubit c initialized to $|0\rangle$. For the subtraction

in Equation (3), we first apply the $\mathsf{UMA}^\dagger$ cells from high to low, update $\texttt{Sign}$ from the carry qubit, and then apply the $\mathsf{MAJ}^\dagger$ cells from low to high to uncompute the carry.

It remains to coherently select the interval $[L, R]$. The endpoint registers storing L and R are obtained from the length registers by reversible affine transformations, which are undone after the arithmetic block. The circuit uses one accumulator bit s, initialized to $|0\rangle$. In the $\mathsf{UMA}^\dagger$ pass, the circuit scans the physical positions from high to low, namely $j = K, K-1, \ldots, k$. For each scanned position j, pruned unary iteration on the endpoint registers coherently produces the two local controls

$$\lambda_j = \texttt{Ctrl} \wedge [R = j], \qquad \mu_j = \texttt{Ctrl} \wedge [L = j].$$

The circuit first updates the accumulator by $s \leftarrow s \oplus \lambda_j$. If the updated value of s is 1, the circuit applies $\mathsf{UMA}^\dagger(c, u_j, v_j)$; otherwise it does nothing on (c, u_j, v_j). After this controlled arithmetic cell, the circuit updates the accumulator by $s \leftarrow s \oplus \mu_j$. Therefore, when the scan reaches $j = R$, the bit λ_j flips s from 0 to 1 before the arithmetic cell, so the $\mathsf{UMA}^\dagger$ cell at $j = R$ is applied. For positions strictly inside the interval, the accumulator remains equal to 1, so the corresponding arithmetic cells are also applied. When the scan reaches $j = L$, the $\mathsf{UMA}^\dagger$ cell is still applied, and then μ_j flips s back to 0. The same update rule is used for every scanned position $j \in [k, K]$. Before the active interval is reached, all arithmetic cells are disabled, so the carry qubit c remains equal to 0 at the interval entrance.

After the $\mathsf{UMA}^\dagger$ pass, c stores the carry bit of the selected subtraction. We update $\texttt{Sign}$ by

$$\texttt{Sign} \leftarrow \texttt{Sign} \oplus c.$$

The $\mathsf{MAJ}^\dagger$ pass visits $j = k, k+1, \ldots, K$. At each position, the accumulator is first updated by $s \leftarrow s \oplus \mu_j$. If the updated value of s is 1, the circuit applies $\mathsf{MAJ}^\dagger(c, u_j, v_j)$. It then updates the accumulator by $s \leftarrow s \oplus \lambda_j$. Therefore the $\mathsf{MAJ}^\dagger$ cells are applied exactly for $j \in [L, R]$, and s and c are restored to $|0\rangle$ at the end.

Figure 10 illustrates the complete pattern for the example $k = 5$ and $K = 7$.

Finally, we explain how the controlled MAJ and UMA cells used above are implemented. We do not add the location-control bit to every gate in the cell; only the Toffoli gate that updates the carry qubit is controlled. Figure 11 shows the corresponding controlled cells.

The gate count consists of the following contributions:

- Generating $[L = j]$ and $[R = j]$ during both scans requires four applications of unary iteration. They use $4(K-k) + O(1)$ Toffoli gates and $8(K-k) + O(1)$ CNOT gates in total.
- Applying one controlled $\mathsf{UMA}^\dagger$ cell at each position in the high-to-low scan uses $4(K-k) + O(1)$ Toffoli gates and $2(K-k) + O(1)$ CNOT gates.
- Applying one controlled $\mathsf{MAJ}^\dagger$ cell at each position in the low-to-high scan uses $3(K-k) + O(1)$ Toffoli gates and $2(K-k) + O(1)$ CNOT gates.
- Computing L and R from ℓ_q and ℓ_s, and subsequently restoring these registers, uses $O(\log_2 n)$ Toffoli and CNOT gates.

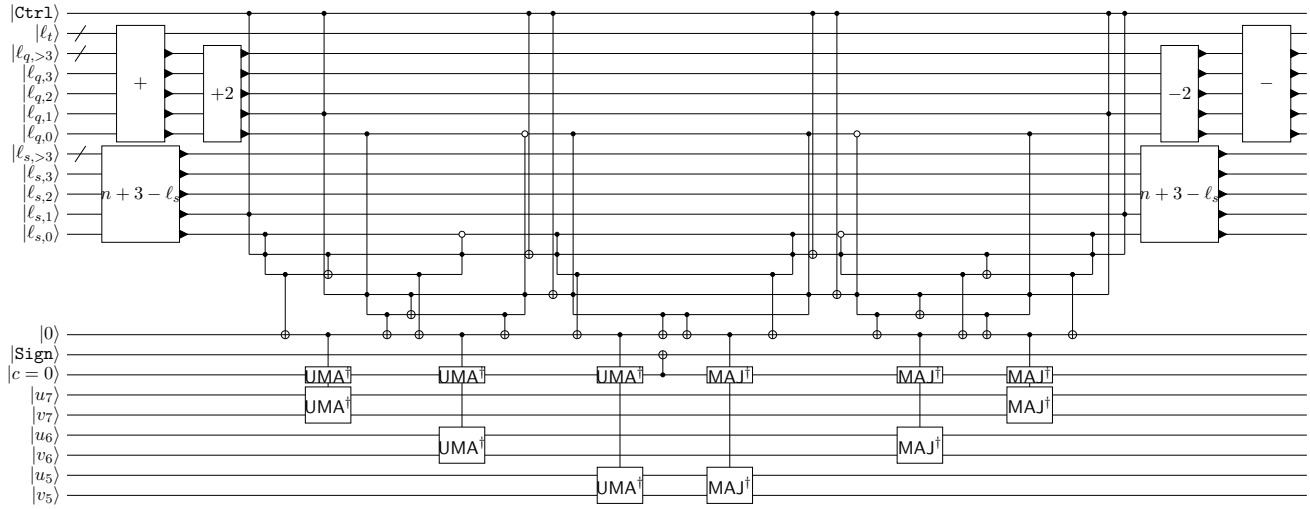
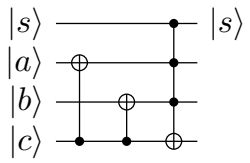
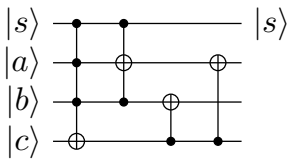


Figure 10: Location-controlled subtraction circuit for $k = 5$ and $K = 7$. The $\mathsf{UMA}^\dagger$ pass computes the carry, `Sign` is updated from c, and the $\mathsf{MAJ}^\dagger$ pass restores the carry qubit.



(a) Controlled MAJ cell.



(b) Controlled UMA cell.

Figure 11: Controlled arithmetic cells used in the location-controlled add/sub circuits.

Adding these terms, the location-controlled `Sub` circuit implementing Equation (3) uses $11(K - k) + O(\log_2 n)$ Toffoli gates and $12(K - k) + O(\log_2 n)$ CNOT gates.

Location-controlled length-update circuit. Finally, we describe the length-update circuit. We focus on updating ℓ_t; the circuit for updating $\ell_{r'}$ is analogous. At the end of one EEA iteration, after the full swap of `Work1` and `Work2`, the left part of `Work1` stores the current value of t, while the left part of `Work2` stores the current value of t'. Let $L_{\text{new}} = \text{len}(t), L_{\text{old}} = \text{len}(t')$. Since the value stored in ℓ_t before the update is L_{old}, it suffices to implement

$$\ell_t \leftarrow \ell_t \oplus L_{\text{old}} \oplus L_{\text{new}}.$$

Thus the length update is reduced to two serial constant-XOR writes: first XOR the old length, and then XOR the new length.

We first describe the subroutine for XORing L_{new} into ℓ_t. This subroutine finds the highest valid nonzero position among the qubits $u_k, \ldots, u_K$. Only positions in the left region not occupied by r' should be inspected. Let $J = n + 3 - \ell_{r'}$ be the right boundary of this valid region with the promise

$k \le \ell_t \le J \le K$ from the active window under consideration. For each $j \in [k, K]$, define the masked bit

$$a_j = \texttt{Ctrl} \wedge [j \le J] \wedge u_j.$$

Also define bits

$$z_j = \bigwedge_{h=j}^{K} \neg a_h.$$

Thus $z_j = 1$ means that no valid nonzero bit occurs among positions $j, j+1, \dots, K$. If the highest valid nonzero position is m, then

$$z_K = \cdots = z_{m+1} = 1, \qquad z_m = \cdots = z_k = 0.$$

If no valid nonzero bit exists in the window, then all z_j are equal to 1.

The highest position can be written using a telescoping XOR. We first XOR the classical constant K into the length register. Then, for $j = K, K-1, \dots, k+1$, whenever $z_j = 1$, we change the current candidate from j to $j-1$ by XORing the classical constant $j \oplus (j-1)$. Finally, if $z_k = 1$, then the entire window contains no valid nonzero bit, and we clear the remaining candidate k by XORing k. Equivalently, the value written by the subroutine is

$$L_{\text{new}} = K \oplus \bigoplus_{j=k+1}^{K} z_j \cdot \big(j \oplus (j-1)\big) \oplus z_k \cdot k,$$

where a term such as $z_j \cdot C$ means that the classical bit string C is XORed into the length register controlled on z_j. If the highest valid nonzero position is m, the expression telescopes to m. If no such position exists, it telescopes to zero.

It remains to implement the constant writes controlled by the bits z_j without storing all of them in clean ancillae. Borrow dirty qubits $g_k, \dots, g_K$, which will be restored at the end. For $j > k$, let V_j denote the Clifford operation that XORs the classical constant $j \oplus (j-1)$ into the length register, and let V_k denote the operation that XORs k. Since each V_j is a product of CNOT gates, $V_j^2 = I$. Assume for the moment that we can perform the transformation

$$g_j \leftarrow g_j \oplus z_j \qquad (j = k, \dots, K).$$

Then the desired controlled write is implemented by placing this transformation between two uses of V_j controlled by the borrowed qubit:

$$V_j^{g_j}, \qquad g_j \leftarrow g_j \oplus z_j, \qquad V_j^{g_j}.$$

Indeed, if the initial value of the borrowed qubit is G_j, then the two controlled writes are

$$V_j^{G_j} V_j^{G_j \oplus z_j} = V_j^{z_j},$$

so the unknown initial value cancels. After all constant writes are completed, we perform the same transformation again to restore every g_j to its original value. Figure 12 illustrates this method for the example $k = 5$ and $K = 7$.

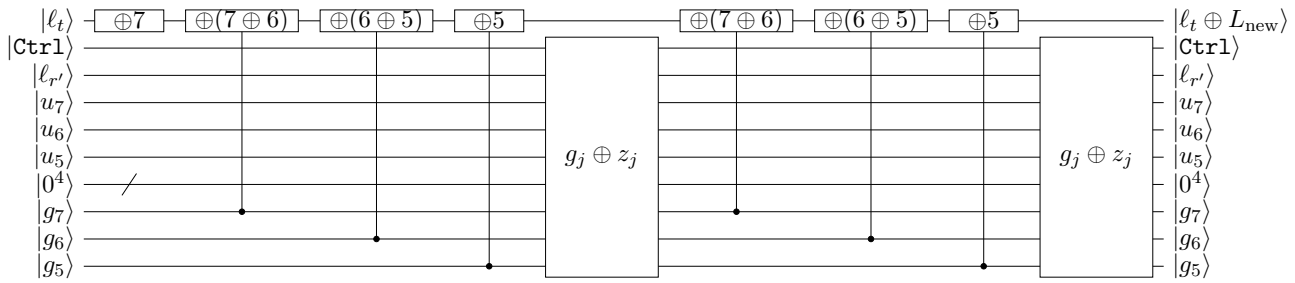


Figure 12: Writing L_{new} using borrowed qubits for $k = 5$ and $K = 7$. The boxes labeled $g_j \oplus z_j$ denote the transformation $g_j \leftarrow g_j \oplus z_j$ for all j in the window.

We now explain how to implement the transformation $g_j \leftarrow g_j \oplus z_j$ for all j. The recurrence is

$$z_K = \neg a_K, \qquad\qquad z_j = \neg a_j \, z_{j+1} \qquad (j < K).$$

To implement this transformation, first apply, for $j = k, k+1, \ldots, K-1$, the Toffoli gates $g_j \leftarrow g_j \oplus (\neg a_j) g_{j+1}$. Then apply the base CNOT gate $g_K \leftarrow g_K \oplus \neg a_K$. Finally apply the same Toffoli gates in reverse order, for $j = K-1, K-2, \ldots, k$. For a fixed $j < K$, the two Toffoli gates acting on g_j are separated by the transformation $g_{j+1} \leftarrow g_{j+1} \oplus z_{j+1}$. Therefore their combined action on g_j is

$$\begin{aligned} g_j &\leftarrow g_j \oplus (\neg a_j) g_{j+1} \oplus (\neg a_j)(g_{j+1} \oplus z_{j+1}) \\ &= g_j \oplus (\neg a_j) z_{j+1} = g_j \oplus z_j. \end{aligned}$$

The bit a_j is produced only locally when the corresponding gate is applied. During a scan of the window, unary iteration on the boundary register supplies the temporary range control

$$b_j = \texttt{Ctrl} \wedge [j \le J].$$

The circuit then computes $a_j = b_j \wedge u_j$ into one clean temporary qubit, uses it as the negative control in the corresponding Toffoli or CNOT, and immediately uncomputes it. The required range controls are supplied by one scan in the order $k, k+1, \ldots, K$ and one scan in the reverse order $K, K-1, \ldots, k$.

Figure 13 shows this transformation for the example $k = 5$ and $K = 7$. The boundary value is computed in place on the $\ell_{r'}$ register, and the temporary bit a is created only when the corresponding update gate is applied.

To update ℓ_t, we invoke this subroutine twice. First we use the analogous construction with v_j in place of u_j and borrow the qubits u_j as the dirty register g_j, which XORs L_{old} into ℓ_t. Then we use the construction described above and borrow the qubits v_j as the dirty register g_j, which XORs L_{new} into ℓ_t. The two calls are serial, and the borrowed dirty qubits are restored after each call.

We now give the explicit resource count. First consider one application of $g_j \leftarrow g_j \oplus z_j$ for all $j \in [k, K]$. Its gate count has three contributions:

- The forward and reverse unary iterations use $2(K-k) + O(1)$ Toffoli gates and $4(K-k) + O(1)$ CNOT gates.

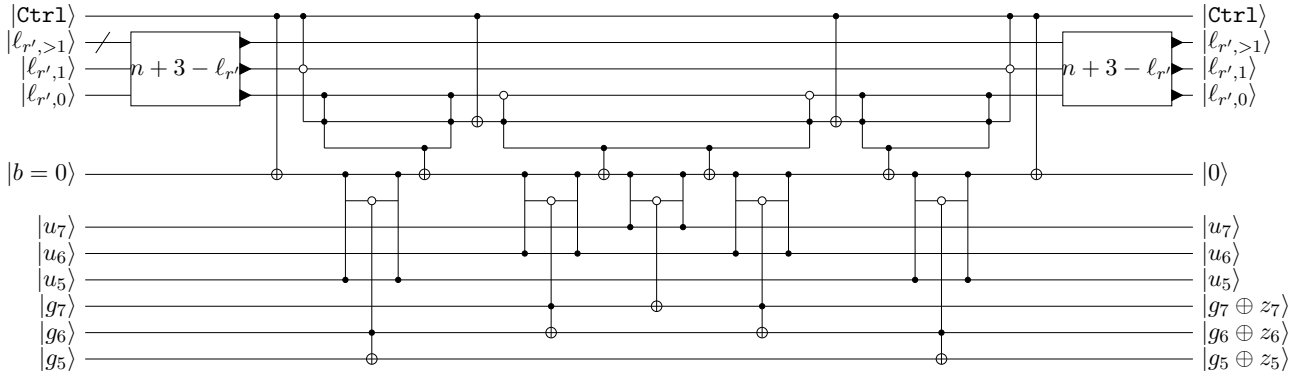


Figure 13: Computing $g_j \leftarrow g_j \oplus z_j$ for $k = 5$ and $K = 7$.

- The forward and reverse applications of $g_j \leftarrow g_j \oplus (\neg a_j) g_{j+1}$ contribute $2(K-k)$ Toffoli gates in total, while the update at $j = K$ contributes one CNOT gate.
- The temporary bit $a_j = b_j \wedge u_j$ is computed once in each direction and uncomputed immediately after use. Over both directions, this uses $2(K-k)+O(1)$ Toffoli gates and $2(K-k)+O(1)$ CNOT gates.

Thus one application of $g_j \leftarrow g_j \oplus z_j$ uses $6(K-k)+O(\log_2 n)$ Toffoli gates and $6(K-k)+O(\log_2 n)$ CNOT gates.

To XOR either L_{old} or L_{new} into ℓ_t, the transformation $g_j \leftarrow g_j \oplus z_j$ is first applied between the two sets of controlled constant XORs and is then applied again to restore the qubits g_j. These two applications use $12(K-k)+O(\log_2 n)$ Toffoli gates and $12(K-k)+O(\log_2 n)$ CNOT gates. Each set of controlled constant XORs uses $2(K-k)+O(\log_2 n)$ CNOT gates and no Toffoli gates. Therefore, XORing one of the two lengths into ℓ_t uses $12(K-k)+O(\log_2 n)$ Toffoli gates and $16(K-k)+O(\log_2 n)$ CNOT gates.

The complete length update performs this procedure once for L_{old} and once for L_{new}. Its total cost is $24(K-k)+O(\log_2 n)$ Toffoli gates and $32(K-k)+O(\log_2 n)$ CNOT gates.

The construction uses exactly $K-k+1$ dirty auxiliary qubits, all of which are restored to their initial states. The same clean auxiliary qubits can be reused throughout the circuit: one stores a_j, one stores b_j, and unary iteration requires at most $\lceil \log_2(K-k) \rceil + 1$ additional clean qubits for $K-k \geq 1$. Thus the construction uses at most $\lceil \log_2(K-k) \rceil + 3$ clean auxiliary qubits.

5 Affine Point Addition with Mid-Circuit Measurement

In the previous sections, we constructed a space-efficient modular inversion circuit based on the extended Euclidean algorithm. The inversion circuit uses two n-qubit field registers together with $O(\log_2 n)$ auxiliary qubits. In this section, we explain how to incorporate this component into an affine point-addition circuit. By allowing mid-circuit measurements and classical feed-forward operations, the affine point-addition block can be implemented using only three n-qubit field registers, plus the $O(\log_2 n)$ auxiliary

qubits required by the EEA inversion subroutine.

The overall circuit is shown in Figure 14. The three main quantum registers are denoted by X, Y, and A, each storing an element of $\mathbb{F}_p$. The remaining $O(\log_2 n)$ EEA ancilla qubits are not shown explicitly in the figure.

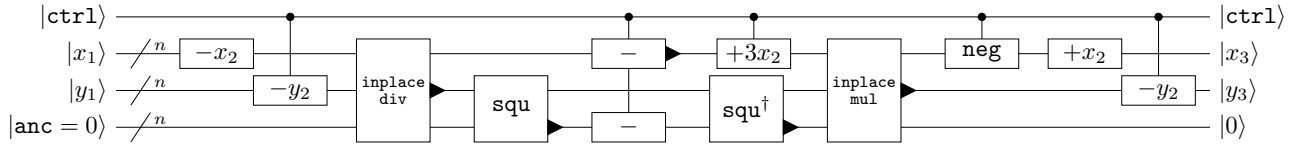


Figure 14: Affine point-addition circuit with mid-circuit measurements using three n-qubit field registers.

The constant additions, subtractions, negations, and squarings in Figure 14 are standard in-place modular operations. The only non-standard components are the in-place division and multiplication subroutines. They have the following abstract interfaces:

$$\text{IDiv}: \quad |x\rangle_X|y\rangle_Y|0\rangle_A \longmapsto |x\rangle_X|y/x\rangle_Y|0\rangle_A, \qquad x \neq 0, \tag{4}$$

$$\text{IMul}: \quad |x\rangle_X|y\rangle_Y|0\rangle_A \longmapsto |x\rangle_X|xy\rangle_Y|0\rangle_A, \qquad x \neq 0. \tag{5}$$

It remains to explain how the in-place maps in Equations (4) and (5) can be implemented with only three field registers. The constructions are similar, so we describe the in-place division subroutine in detail. Let S denote the $O(\log_2 n)$ auxiliary register used by the EEA inversion circuit. The goal is to implement

$$|x\rangle_X|y\rangle_Y|0\rangle_A|0\rangle_S \longmapsto |x\rangle_X|y/x\rangle_Y|0\rangle_A|0\rangle_S, \qquad x \neq 0.$$

First, run the forward EEA inversion circuit on X, using A as the large clean workspace and S as the small auxiliary register:

$$|x\rangle_X|y\rangle_Y|0\rangle_A|0\rangle_S \longmapsto |x^{-1}\rangle_X|y\rangle_Y|0\rangle_A|\Gamma(x)\rangle_S.$$

Here $|\Gamma(x)\rangle_S$ denotes the intermediate EEA state that is kept until the inverse EEA circuit is applied. Next, use a standard modular multiplication to write the quotient into A:

$$|x^{-1}\rangle_X|y\rangle_Y|0\rangle_A \longmapsto |x^{-1}\rangle_X|y\rangle_Y|y/x\rangle_A.$$

At this point A is no longer clean, and therefore cannot be used as the large workspace for the inverse EEA circuit. Instead, we recycle the register Y by measurement. Apply $H^{\otimes n}$ to Y and measure it in the computational basis. If the classical outcome is $b \in \{0,1\}^n$, the remaining quantum state acquires the phase $(-1)^{b\cdot y}$, where the dot product is over $\mathbb{F}_2$. The measured register Y is then reset to $|0\rangle$.

Now Y is a clean n-qubit register, so it can be used as the large workspace for the inverse EEA circuit. Applying the inverse EEA circuit gives

$$(-1)^{b\cdot y}|x^{-1}\rangle_X|0\rangle_Y|y/x\rangle_A|\Gamma(x)\rangle_S \longmapsto (-1)^{b\cdot y}|x\rangle_X|0\rangle_Y|y/x\rangle_A|0\rangle_S. \tag{6}$$

The phase in Equation (6) is removed by recomputing the measured value y from x and y/x. Specifically, apply the modular multiplication

$$|x\rangle_X|0\rangle_Y|y/x\rangle_A \longmapsto |x\rangle_X|y\rangle_Y|y/x\rangle_A, \tag{7}$$

then apply the classically controlled Pauli correction

$$Z^b = Z^{b_0} \otimes Z^{b_1} \otimes \cdots \otimes Z^{b_{n-1}}$$

to the register Y. This contributes another factor $(-1)^{b\cdot y}$ and therefore cancels the measurement phase. Finally, reverse the multiplication in Equation (7), obtaining

$$|x\rangle_X|0\rangle_Y|y/x\rangle_A.$$

Swapping Y and A completes the in-place division:

$$|x\rangle_X|y/x\rangle_Y|0\rangle_A.$$

The corresponding circuit is shown in Figure 15.

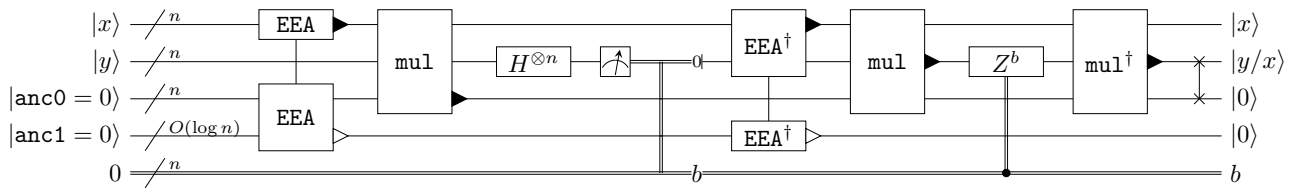


Figure 15: In-place division with mid-circuit measurement and classical feed-forward operations.

The in-place multiplication subroutine is implemented by the same mechanism. One first computes xy into the clean register A, measures and resets the old $Y = y$ register, uses the remaining registers to recompute y for the Z^b phase correction, uncomputes this recomputation, and finally swaps Y with A. Thus both IDiv and IMul require only the three field registers X, Y, A, together with the $O(\log_2 n)$ EEA auxiliary register S. Therefore the affine point-addition circuit in Figure 14 uses $3n + O(\log_2 n)$ logical qubits.

6 Resource Estimation and the ECDLP

We adopt the resource-estimation methodology introduced by Roetteler et al. [RNSL17]. In Shor's algorithm for the ECDLP, the dominant quantum cost arises from the sequence of controlled elliptic-curve point additions performed in superposition. We consider the point additions in affine coordinates over the finite field $\mathbb{F}_p$, where p is an n-bit prime. At this level, point addition reduces to a fixed sequence of modular additions, subtractions, multiplications, squarings, and inversions. Among these operations, modular inversion typically dominates both the space complexity and the Toffoli gate count.

6.1 Space complexity improvements

In this work, we focus exclusively on reducing the space complexity required for modular inversion, rather than exploring trade-offs among Toffoli count, circuit depth, and space complexity as previous works [RNSL17, HJN+20, BZG+26, Sch26]. Specifically, we present an exact, space-efficient, in-place reversible quantum circuit for modular inversion that implements Equation (1). The circuit is based on the register-sharing technique originally proposed by Proos and Zalka [PZ03]. This approach reduces the nunmber of qubits required for modular inversion to

$$2n + 6\lfloor \log_2 n \rfloor + 19.$$

Of this total, $2n + 4\lfloor \log_2 n \rfloor + 15$ qubits correspond to the base circuit shown in Figures 5 and 6, while the remaining $2\lfloor \log_2 n \rfloor + 4$ qubits are required for the unary iteration described in Section 4.3.

For elliptic curve point addition shown in Figure 14, the overall space complexity is then

$$3n + 6\lfloor \log_2 n \rfloor + 19.$$

6.2 Asymptotic gate counts

Modular inversion. We first derive the asymptotic Toffoli and CNOT gate counts of the proposed modular inversion circuit. Since the circuit consists of several types of location-controlled operations, we first recall (in Section 4.4) the gate cost incurred by each circuit block at algorithmic step T. These per-step costs are summarized in Table 5. We then combine them with the active-window analysis developed in Section 4.2 to obtain the overall asymptotic gate complexity.

Circuit block on step T	**Toffoli cost**	**CNOT cost**
Location-controlled addition/subtraction on r's	$11(K_1(T) - k_1(T))$	$12(K_1(T) - k_1(T))$
Location-controlled swap	$2(K_2(T) - k_2(T))$	$4(K_2(T) - k_2(T))$
Location-controlled addition/subtraction on t's	$9(K_3(T) - k_3(T))$	$10(K_3(T) - k_3(T))$
Length-update circuit for ℓ_t (only on $4 \mid T$)	$24(K_4(T) - k_4(T))$	$32(K_4(T) - k_4(T))$
Length-update circuit for $\ell_{r'}$ (only on $4 \mid T$)	$24(K_5(T) - k_5(T))$	$32(K_5(T) - k_5(T))$

Table 5: Per-step Toffoli and CNOT gate counts of the major circuit blocks in the modular inversion algorithm.

- **Location-controlled addition/subtraction on r's.** These operations cost

$$2\sum_{T=1}^{4\lceil cn \rceil} 11\big(K_1(T) - k_1(T)\big) + O(n \log_2 n) = (44c + 11)n^2 + O(n \log_2 n)$$

  Toffoli gates, and $(48c + 12)n^2 + O(n \log_2 n)$ CNOT gates.

- **Location-controlled swap.** These operations cost

$$\sum_{T=1}^{4\lceil cn\rceil} 2\big(K_2(T) - k_2(T)\big) + O(n\log_2 n) = (4c+1)n^2 + O(n\log_2 n)$$

Toffoli gates, and $(8c+2)n^2 + O(n\log_2 n)$ CNOT gates.

- **Location-controlled addition/subtraction on t's.** These operations cost

$$2\sum_{T=1}^{4\lceil cn\rceil} 9\big(K_3(T) - k_3(T)\big) + O(n\log_2 n) = (72c-36)n^2 + O(n\log_2 n)$$

Toffoli gates, and $(80c-40)n^2 + O(n\log_2 n)$ CNOT gates.

- **Length-update circuit for updating ℓ_t.** We execute the length-update circuit only at step indices T that are multiples of four. These operations cost

$$\sum_{T'=1}^{\lceil cn\rceil} 24\big(K_4(4T') - k_4(4T')\big) + O(n\log_2 n) = 12cn^2 + O(n\log_2 n).$$

Toffoli gates, and $16cn^2 + O(n\log_2 n)$ CNOT gates.

- **Length-update circuit for updating $\ell_{r'}$.** We execute the length-update circuit only at step indices T that are multiples of four. These operations cost

$$\sum_{T'=1}^{\lceil cn\rceil} 24\big(K_4(4T') - k_4(4T')\big) + O(n\log_2 n) = (12c-12)n^2 + O(n\log_2 n).$$

Toffoli gates, and $(16c-16)n^2 + O(n\log_2 n)$ CNOT gates.

In addition to the circuit blocks analyzed above, the modular inversion circuit (illustrated in Figures 5 and 6) contains four controlled Shift circuits. Each Shift circuit consists of $n + O(1)$ Toffoli gates and $2n + O(1)$ CNOT gates. Therefore, these four Shift circuits contribute an additional $16cn^2 + O(n)$ Toffoli gates and $32cn^2 + O(n)$ CNOT gates.

Combining all contributions, the total number of Toffoli gates required by our (one-sided without reversion) modular inversion circuit grows asymptotically as

$$(160c-36)n^2 + O(n\log_2 n) < 216.636n^2 + O(n\log_2 n).$$

The asymptotic bound of CNOT gates required is

$$(200c-42)n^2 + O(n\log_2 n) < 273.795n^2 + O(n\log_2 n).$$

Point addition. As discussed in Section 5, we count one affine point addition with mid-circuit measurement as consisting of 1 in-place division, 1 in-place multiplication, 2 modular squarings, and a constant number of modular additions/subtractions/negations. The corresponding circuit is shown schematically in Figure 14. Among these modular arithmetic operations, the modular additions/subtractions/negations contribute only $O(n \log_2 n)$ Toffoli gates in total and are therefore absorbed into the lower-order term. The in-place division, as well as the in-place multiplication, consists of 2 modular divisions and 3 modular multiplications as shown in Figure 15.

We use an improved modular multiplication/squaring circuit described in Appendix B, which costs $17n^2 + O(n \log_2 n)$ Toffoli gates. Therefore, a single affine point addition with mid-circuit measurement can be implemented by at most

$$(4 \cdot 216.636 + 6 \cdot 17 + 2 \cdot 17)n^2 + O(n \log_2 n) < 1003n^2 + O(n \log_2 n)$$

Toffoli gates.

6.3 Numerical experiments

All resource-estimation experiments were conducted on a Linux server equipped with an AMD EPYC 7302 processor (up to 3.0 GHz). Circuit generation, compilation and optimization were performed using IBM Qiskit. The constructed circuits were transpiled into a gate basis consisting of `CCX`, `CX`, and `X` gates, in order to obtain accurate Toffoli-gate and CNOT-gate counts.

The implementation follows the explicit circuit constructions in Sections 4 and 5. Since modular inversion is the most involved subroutine and a main contributor to the point-addition cost, we report its implementation separately.

- **Modular-inversion subroutine.** We implement the one-step EEA circuit as a collection of Qiskit modules. These modules include the pre-shift and post-shift blocks, the location-controlled arithmetic and swap blocks, the phase-update logic, and the length-update circuits. A full inversion instance is generated by following the step schedule of Algorithm 3. Each step instantiates only its active window. The step circuits are counted in chunks and are then combined with the shared-register EEA wrapper.
- **Point-addition circuit.** We implement the point-addition circuit blockwise. Figure 14 specifies the top level affine point addition schedule, including the constant coordinate updates, the in-place division and multiplication calls, the square-minus block, and the final corrections. The point coordinates x_2 and y_2 are treated as classically precomputed constants. The in-place division and multiplication blocks are assembled according to Figure 15, from the shared-register EEA circuit, modular multiplication blocks, inverse multiplication blocks, and the explicit mid-circuit measurements and feed-forward phase corrections shown in the figure.

To obtain the compiled gate counts of these implemented circuits, we use a layered blockwise counting pipeline. This is necessary because directly expanding the full point-addition circuit into a single Qiskit circuit over elementary operations is impractical. We first recursively decompose and count reusable

Qiskit subblocks, including constant-coordinate updates, controlled modular additions and subtractions, modular doubling and halving, and the shared-register EEA wrapper. We then assemble larger arithmetic blocks from these counted components with their exact multiplicities. For example, a modular multiplication block contains n controlled modular additions and $n-1$ modular doublings, while its inverse contains n controlled modular subtractions and $n-1$ modular halvings. The squaring block reuses the same multiplication count and adds the explicit CNOT cost for copy and uncompute. This yields a complete compiled gate count for the point-addition circuit while preserving the block multiplicities.

In addition to compiling and counting the reusable subblocks, we apply a local template-based optimization before each subblock count is stored and reused. The optimizer performs local cancellation and Qiskit template replacement on small reversible segments. To make the template identities directly applicable, we restrict these segments to the gate library $\{X, \mathrm{CX}, \mathrm{CCX}\}$. A rewritten segment is accepted only if its gate count does not increase.

To improve efficiency and preserve the circuit structure, we apply the optimization segment by segment. We scan each decomposed subblock linearly and collect consecutive X, CX, and CCX gates into one reversible segment. When the scan reaches any other instruction, the current segment is optimized, the out-of-library instruction is counted directly, and a new segment is started. Thus, in circuits with measurements and feed-forward phase corrections, these operations are not rewritten. They simply separate the neighboring reversible segments. If a segment exceeds the runtime limits, it is counted unchanged.

To validate the implementation, we developed test suites for both the modular-inversion and point-addition circuits. The tests include representative examples over prime fields with p ranging from small values such as $p = 3$ to 512-bit primes, and with circuit widths n ranging up to 512. They additionally verify register layouts, circuit schedules, arithmetic sub-blocks, and compiled block assembly.

The corresponding circuit implementation is included in our open-source codebase.[2] For each problem size n, we generated the full circuit instance up to the theoretical upper bound on the number of cycles derived in this work, transpiled it into the `CCX`, `CX`, and `X` basis, and recorded the resulting gate counts. We report results for $n \in \{64, 128, 160, 192, 224, 256, 384, 512\}$. The numerical values are listed in Table 6.

n	Modular inversion		Point addition	
	Toffoli ($\times 10^6$)	CNOT ($\times 10^6$)	Toffoli ($\times 10^6$)	CNOT ($\times 10^6$)
64	2.41	3.32	5.39	7.98
128	8.47	11.82	19.17	28.97
160	12.63	17.80	28.75	43.92
192	17.70	25.04	40.41	62.05
224	23.57	33.43	53.96	83.99
256	30.59	43.39	70.10	108.09
384	65.97	94.31	152.00	237.44
512	115.81	165.63	267.27	416.32

Table 6: Gate counts for modular inversion and point addition circuits over n-bit prime fields.

[2] GitHub repository: https://github.com/ZeroWang030221/Space-Efficient-Quantum-Algorithm-for-Elliptic-Curve-Discrete-Logarithms-with-Resource-Estimation

6.4 Resource estimates for Shor's ECDLP circuit

For the full ECDLP attack, we do not repeat the controlled point-addition circuit bit-by-bit for all $2n$ control bits. Instead, we follow the signed-window technique of [HJN+20, Section 5.1]. Namely, we partition the $2n$ control bits arising in the double-scalar multiplication into windows of size w, and implement each window by one signed windowed point addition.

In the signed-window construction, one of the w bits is used as a sign bit, while the remaining $w-1$ bits address a precomputed table of point multiples. As shown schematically in Fig. 10 of [HJN+20], one such windowed point addition uses the arithmetic core of an affine Weierstrass point addition. The coordinates of the selected precomputed point are obtained through three table queries, and its sign is handled by one modular negation. In our space-optimized circuit, these three queries are implemented by five sequential table look-ups.[3] Since the dominant n^2 term in our Toffoli accounting comes from modular divisions, multiplications and squaring, replacing the classically known addend by an addend obtained from the look-up table does not change the asymptotic scaling.

To obtain a clean asymptotic bound, we choose window size $w = 2\log_2 n$. The five sequential table look-ups contribute $5 \cdot 2^w$ Toffoli gates to each windowed point addition. The full double-scalar multiplication uses approximately $2n/w$ windows, so its dominant Toffoli count becomes

$$\frac{2n}{w}\left(1003n^2 + 5 \cdot 2^w + O(n\log_2 n)\right) = 1008\frac{n^3}{\log_2 n} + O(n^2).$$

For $n = 256$, following the analysis in [BZG+26], the Toffoli count of the complete Shor ECDLP circuit is estimated as

$$28 \cdot \left(5 \cdot 2^{16} + Q_A\right). \tag{8}$$

Here, Q_A denotes the Toffoli count of a single controlled affine point-addition circuit. For a window size of $w = 16$, the number of signed-window point additions is $\frac{2n}{w} - 4 = 28$. The five sequential table look-ups described above incur an additional $5 \cdot 2^w$ Toffoli gates per window, which becomes $5 \cdot 2^{16}$ when $w = 16$. Substituting our concrete value of $Q_A(256)$ shows that our algorithm requires $2^{30.63}$ Toffoli gates for the curve secp256k1.

Finally, we note that compression techniques and multi-run tradeoffs [Eke19] could potentially yield further improvements to the constant factors in our overall resource estimates.

Acknowledgments

We thank Pierre-Luc Dallaire-Demers for pointing out an error in an earlier version of the EEA step bound proof, which we have replaced with a matrix–continuant proof based on a finite polyhedral antinorm certificate; see Appendix A.1. This work was supported by the Innovation Program for Quantum Science and Technology (Grant Number 2024ZD0300500). ZY, ZW, YS and TL are supported by the National Natural Science Foundation of China (Grant Numbers 62372006 and 92365117).

[3] The circuit first queries (x_2, y_2), then x_2, and finally (x_2, y_2). If two free n-qubit registers were available, the two coordinates in each (x_2, y_2) query could be loaded together. Since only one free n-qubit register is available in our space-optimized circuit, each (x_2, y_2) query is split into two sequential look-ups, giving $2 + 1 + 2 = 5$ look-ups in total.

References

[AMMR13] Matthew Amy, Dmitri Maslov, Michele Mosca, and Martin Roetteler. A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 32(6):818–830, 2013. https://doi.org/10.1109/TCAD.2013.2244643.

[BBvHL20] Gustavo Banegas, Daniel J. Bernstein, Iggy van Hoof, and Tanja Lange. Concrete quantum cryptanalysis of binary elliptic curves. *Cryptology ePrint Archive*, 2020. https://eprint.iacr.org/2020/1296.

[BD20] Elaine Barker and Quynh Dang. NIST special publication 800-57 part 1, revision 5, recommendation for key management: Part 1–general. *NIST, Technical Report*, page 171, 2020.

[BWBG+06] Simon Blake-Wilson, Nelson Bolyard, Vipul Gupta, Chris Hawk, and Bodo Moeller. Elliptic curve cryptography (ECC) cipher suites for transport layer security (TLS), 2006. https://www.rfc-editor.org/rfc/rfc4492.html.

[BWQ99] Simon Blake-Wilson and M. Qu. Standards for efficient cryptography (sec) 2: Recommended elliptic curve domain parameters. *Certicom Research, Oct*, page 25, 1999.

[BZG+26] Ryan Babbush, Adam Zalcman, Craig Gidney, Michael Broughton, Tanuj Khattar, Hartmut Neven, Thiago Bergamaschi, Justin Drake, and Dan Boneh. Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations, 2026. arXiv:2603.28846

[CDKM04] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. A new quantum ripple-carry addition circuit. *arXiv preprint*, 2004. arXiv:quant-ph/0410184

[CFS25] Clémence Chevignard, Pierre-Alain Fouque, and André Schrottenloher. Reducing the number of qubits in quantum factoring. In *Annual International Cryptology Conference*, pages 384–415. Springer, 2025. https://doi.org/10.1007/978-3-032-01878-6_13.

[CFS26] Clémence Chevignard, Pierre-Alain Fouque, and André Schrottenloher. Reducing the number of qubits in quantum discrete logarithms on elliptic curves. Cryptology ePrint Archive, Paper 2026/280, 2026. https://eprint.iacr.org/2026/280.

[CvD10] Andrew M. Childs and Wim van Dam. Quantum algorithms for algebraic problems. *Reviews of Modern Physics*, 82(1):1–52, 2010. https://doi.org/10.1103/RevModPhys.82.1.

[DH76] Whitfield Diffle and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6), 1976. https://doi.org/10.1145/3549993.3550007.

[Eke19] Martin Ekerå. Revisiting Shor's quantum algorithm for computing general discrete logarithms, 2019. arXiv:1905.09084

[ElG85] Taher ElGamal. A public key cryptosystem and a signature scheme based on discrete logarithms. *IEEE Transactions on Information Theory*, 31(4):469–472, 1985. https://doi.org/10.1109/TIT.1985.1057074.

[FMMC12] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3):032324, 2012. https://doi.org/10.1103/PhysRevA.86.032324.

[GE21] Craig Gidney and Martin Ekerå. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021. https://doi.org/10.22331/q-2021-04-15-433.

[GG16] Steven D. Galbraith and Pierrick Gaudry. Recent progress on the elliptic curve discrete logarithm problem. *Designs, Codes and Cryptography*, 78(1):51–72, 2016. https://doi.org/10.1007/s10623-015-0146-7.

[Gid25] Craig Gidney. How to factor 2048 bit RSA integers with less than a million noisy qubits. *arXiv preprint*, 2025. arXiv:2505.15917

[GN96] Robert B. Griffiths and Chi-Sheng Niu. Semiclassical Fourier transform for quantum computation. *Physical Review Letters*, 76(17):3228, 1996. https://doi.org/10.1103/PhysRevLett.76.3228.

[GYCM25] Quan Gu, Han Ye, Junjie Chen, and Xiongfeng Ma. Resource analysis of Shor's elliptic curve algorithm with an improved quantum adder on a two-dimensional lattice. *arXiv preprint arXiv:2510.23212*, 2025.

[Has36] Helmut Hasse. Zur theorie der abstrakten elliptischen Funktionenkörper II. Automorphismen und Meromorphismen. Das Additionstheorem. *Journal für die reine und angewandte Mathematik*, 1936. https://doi.org/10.1515/crll.1936.175.69.

[HJN+20] Thomas Häner, Samuel Jaques, Michael Naehrig, Martin Roetteler, and Mathias Soeken. Improved quantum circuits for elliptic curve discrete logarithms. In *International Conference on Post-quantum Cryptography*, pages 425–444. Springer, 2020. https://doi.org/10.1007/978-3-030-44223-1_23.

[HRS17] Thomas Häner, Martin Roetteler, and Krysta M. Svore. Factoring using $2n + 2$ qubits with Toffoli based modular multiplication. *Quantum Info. Comput.*, 17(7–8):673–684, June 2017. https://dl.acm.org/doi/abs/10.5555/3179553.3179560.

[JKL+22] Kyungbae Jang, Wonwoong Kim, Sejin Lim, Yeajun Kang, Yujin Yang, and Hwajeong Seo. Optimized implementation of quantum binary field multiplication with Toffoli depth one. In *International Conference on Information Security Applications*, pages 251–264. Springer, 2022. https://doi.org/10.1007/978-3-031-25659-2_18.

[JMV01] Don Johnson, Alfred Menezes, and Scott Vanstone. The elliptic curve digital signature algorithm (ECDSA). *International Journal of Information Security*, 1(1):36–63, 2001. https://doi.org/10.1007/s102070100002.

[JSB+25] Kyungbae Jang, Vikas Srivastava, Anubhab Baksi, Santanu Sarkar, and Hwajeong Seo. New quantum cryptanalysis of binary elliptic curves (extended version). *Cryptology ePrint Archive*, 2025. https://eprint.iacr.org/2025/017.

[KJW+26] Hyunji Kim, Kyungbae Jang, Siyi Wang, Anubhab Baksi, Gyeongju Song, Hwajeong Seo, and Anupam Chattopadhyay. New quantum circuits for ECDLP: Breaking prime elliptic curve cryptography in minutes. Cryptology ePrint Archive, Paper 2026/106, 2026. https://eprint.iacr.org/2026/106.

[KMRVvK25] Gregory D Kahanamoku-Meyer, Seyoon Ragavan, Vinod Vaikuntanathan, and Katherine van Kirk. The Jacobi factoring circuit: Quantum factoring with near-linear gates and sublinear space and depth. In *Proceedings of the 57th Annual ACM Symposium on Theory of Computing*, pages 1496–1507, 2025. https://doi.org/10.1145/3717823.3718273.

[Kob87] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of Computation*, 48(177):203–209, 1987. https://doi.org/10.1090/S0025-5718-1987-0866109-5.

[KSG+25] Tanuj Khattar, Noah Shutty, Craig Gidney, Adam Zalcman, Noureldin Yosri, Dmitri Maslov, Ryan Babbush, and Stephen P. Jordan. Verifiable quantum advantage via optimized DQI circuits, 2025. arXiv:2510.10967

[Mil85] Victor S. Miller. Use of elliptic curves in cryptography. In *Conference on the Theory and Application of Cryptographic Techniques*, pages 417–426. Springer, 1985. https://doi.org/10.1007/3-540-39799-X_31.

[NB08] Satoshi Nakamoto and A. Bitcoin. A peer-to-peer electronic cash system. *Bitcoin.–URL: https://bitcoin. org/bitcoin. pdf*, 4(2):15, 2008.

[NC10] Michael A. Nielsen and Isaac L. Chuang. *Quantum computation and quantum information*. Cambridge University Press, 2010.

[PWLK22] Dedy Septono Catur Putranto, Rini Wisnu Wardhani, Harashta Tatimma Larasati, and Howon Kim. Another concrete quantum cryptanalysis of binary elliptic curves. *Cryptology ePrint Archive*, 2022. https://eprint.iacr.org/2022/501.

[PZ03] John Proos and Christof Zalka. Shor's discrete logarithm quantum algorithm for elliptic curves. *Quantum Info. Comput.*, 3(4):317–344, July 2003. https://dl.acm.org/doi/abs/10.5555/2011528.2011531.

[Reg25] Oded Regev. An efficient quantum factoring algorithm. *Journal of the ACM*, 72(1):1–13, 2025. https://doi.org/10.1145/3708471.

[RNSL17] Martin Roetteler, Michael Naehrig, Krysta M. Svore, and Kristin Lauter. Quantum resource estimates for computing elliptic curve discrete logarithms. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 241–270. Springer, 2017. https://doi.org/10.1007/978-3-319-70697-9_9.

[RSA78] Ronald L. Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. https://doi.org/10.1145/359340.359342.

[RV24] Seyoon Ragavan and Vinod Vaikuntanathan. Space-efficient and noise-robust quantum factoring. In *Annual International Cryptology Conference*, pages 107–140. Springer, 2024. https://doi.org/10.1007/978-3-031-68391-6_4.

[SG09] Douglas Stebila and Jon Green. Elliptic curve algorithm integration in the secure shell transport layer, 2009. https://www.rfc-editor.org/rfc/rfc5656.

[Sho94] Peter W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE, 1994. https://doi.org/10.1109/SFCS.1994.365700.

[TT23] Ren Taguchi and Atsushi Takayasu. Concrete quantum cryptanalysis of binary elliptic curves via addition chain. In *Cryptographers' Track at the RSA Conference*, pages 57–83. Springer, 2023. https://doi.org/10.1007/978-3-031-30872-7_3.

[BBG+18] Ryan Babbush, Craig Gidney, Dominic W. Berry, Nathan Wiebe, Jarrod McClean, Alexandru Paler, Austin Fowler, and Hartmut Neven. Encoding electronic spectra in quantum circuits with linear T complexity. *Physical Review X*, 8:041015, 2018. https://doi.org/10.1103/PhysRevX.8.041015.

[Gid19] Craig Gidney. Spooky pebble games and irreversible uncomputation. https://algassert.com/post/1905, 2019. Blog post, accessed June 26, 2026.

[KH21] Panjin Kim and Daewan Han. Measurement-based uncomputation applied to controlled modular multiplication. 2102.01453, 2021.

[KSS25] Niels Kornerup, Jonathan Sadun, and David Soloveichik. Tight bounds on the spooky pebble game: Recycling qubits with measurements. *Quantum*, 9:1636, 2025. https://doi.org/10.22331/q-2025-02-18-1636.

[LMNS25] Alessandro Luongo, Antonio Michele Miti, Varun Narasimhachar, and Adithya Sireesh. Measurement-based uncomputation of quantum circuits for modular arithmetic. In *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, pages 1–7. IEEE, 2025. https://doi.org/10.1109/DAC63849.2025.11132981.

[Gid25] Craig Gidney. A classical-quantum adder with constant workspace and linear gates. *arXiv preprint*, 2025. arXiv:2507.23079

[Sch26] André Schrottenloher. Optimized point addition circuits for elliptic curve discrete logarithms. *arXiv preprint*, 2026. arXiv:2606.02235

[NV+25] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex

Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. arXiv:2506.13131

[GP13] Nicola Guglielmi and Vladimir Protasov. Exact computation of joint spectral characteristics of linear operators. *Foundations of Computational Mathematics*, 13(1):37–97, 2013. https://doi.org/10.1007/s10208-012-9121-0.

[YLT+26] Haotian Ye, Haowei Lin, Jingyi Tang, Yizhen Luo, Rahul Thapa, Caiyin Yang, Chang Su, Rui Yang, Ruihua Liu, Rundao Li, Zeyu Li, Pengwei Sun, Chong Gao, Dachao Ding, Guangrong He, Miaolei Zhang, Lina Sun, Wenyang Wang, Yuchen Zhong, Zhuohao Shen, Puheng Li, Pan Lu, Bianxiao Cui, Di He, Jianzhu Ma, Junfeng Li, Hexi Baoyin, Yejin Choi, Stefano Ermon, Xiaowen Chu, Tongyang Li, Yuzhi Xu, and James Zou.

Structured scaling of AI discovery across diverse scientific domains.

arXiv preprint arXiv:2604.19341, 2026.

https://arxiv.org/abs/2604.19341.

A Proof Details

A.1 Bounds on the total number of steps

Suppose that after m iterations of the Extended Euclidean Algorithm, we obtain $r_m = 1$ and $r_{m+1} = 0$, with intermediate quotients $q_1, q_2, \ldots, q_m$, where $q_i = \lfloor r_{i-1}/r_i \rfloor$ for $i = 1, 2, \ldots, m$. Then the total number of steps required in our four-phase algorithm can be expressed as

$$N = 4 \sum_{i=1}^{m} \left(\lfloor \log_2 q_i \rfloor + 1 \right). \tag{9}$$

In this appendix, we prove that $4n \le N \le 4\lceil cn \rceil$, where $c = 3/\log_2(2+\sqrt{3})$.

The lower bound. The lower bound of N can be derived directly. Since $r_{i-1} = q_i r_i + r_{i+1} < (q_i+1) r_i$, it follows that

$$N \ge 4 \sum_{i=1}^{m} \log_2(q_i + 1) > 4 \sum_{i=1}^{m} \log_2 \frac{r_{i-1}}{r_i} = 4 \log_2 p \ge 4(n-1).$$

Because N must be a multiple of 4, we conclude that $N \ge 4n$.

Asymptotic sharpness of the upper bound. Before we moving on to the proof of the upper bound, we first express why the constant $c = 3/\log_2(2+\sqrt{3})$ in the upper bound $N \le 4\lceil cn \rceil$ is asymptotically optimal. We consider a different roadmap from deriving the lower bound: instead of computing the upper bound of N, we fix the total number of steps N and then compute the minimum possible value of p.

For integer $q \geq 1$, define the EEA transformer matrix $M(q) := \begin{pmatrix} q & 1 \\ 1 & 0 \end{pmatrix}$. The EEA iteration can therefore be written as $(r_{i-1}, r_i)^T = M(q_i)(r_i, r_{i+1})^T$. Iterating this identity and using $(r_m, r_{m+1}) = (1, 0)$ gives

$$\begin{pmatrix} p \\ x \end{pmatrix} = M(q_1) \cdots M(q_m) \begin{pmatrix} 1 \\ 0 \end{pmatrix} = M(q_1) \cdots M(q_{m-1}) \begin{pmatrix} q_m \\ 1 \end{pmatrix}. \tag{10}$$

In particular,

$$p = K(q_1, \ldots, q_m) := e_1^T M(q_1) \ldots M(q_m) e_1. \tag{11}$$

Given equation (11), writing $d(q) := \lfloor \log_2 q \rfloor + 1$, our problem is to find a slowest growth pattern for products of the matrices $M(q)$ with a fixed weight sum $N/4 = \sum_{i=1}^{m} (d(q_i) + 1)$, where a factor $M(q)$ has weight $d(q)$. Experiments for small total quotient bit-lengths suggest that, with $\sum d(q)$ fixed, the quotient sequence $(1, 2)$ has the minimum averaged growing rate, i.e. the nearly alternating quotient sequence $(2, 1, 2, 1, \ldots, 2, 1, 2, 2)$ asymptotically minimizes the resulting value of p.

Since p is the upper-left and largest entry of $P := M(q_1) \cdots M(q_m)$, we have $p \leq \rho(P) \leq \|P\|_\infty \leq 2p$, here $\rho(P)$ denotes the spectral radius, i.e. the maximum absolute value of eigenvalues, and $\|P\|_\infty$ denotes the maximum row sum. So p has the same order of magnitude with the spectral radius of the matrix product P. Therefore, the repeated block $(1, 2)$ in the quotient sequence suggests the asymptotic growth rate: it has total weight 3, while $M(1)M(2) = \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix}$ has spectral radius $2 + \sqrt{3}$. This motivates the normalization $\rho := 2 + \sqrt{3}$ and $\lambda := \rho^{1/3}$, so that one unit of weight corresponds to a factor λ.

We then give the formal argument on the aymptotic sharpness of the constant c. For $k \geq 0$, consider a quotient sequence

$$Q_k = \big(2, \underbrace{1, 2, 1, 2, \ldots, 1, 2}_{k \text{ copies of } (1,2)}, 2\big),$$

its corresponding step number is $N_k = 12k + 16$. Let $M := M(1)M(2) = \begin{pmatrix} 3 & 1 \\ 2 & 1 \end{pmatrix}$ be a two-iteration EEA transformation matrix with maximal eigenvalue $\rho = 2 + \sqrt{3}$. From basic linear algebra argument, the value of $p = e_1^T M(2) M^k M(2) e_1 = (2, 1) M^k (2, 1)^T$ will asymptotically be bounded by $\Theta(\rho^k)$. Consequently, $\log_2 p_k = k \log_2 \rho + O(1)$, and

$$\lim_{k \to \infty} \frac{N_k}{\log_2 p_k} = \frac{12}{\log_2 \rho} = 4c.$$

Therefore, the coefficient c in the upper bound for $N \leq 4\lceil cn \rceil$ cannot be decreased for general admissible quotient sequences.

In the rest of this section, we will discuss the roadmap for proving this sharp upper bound, and then give the formal proof.

Roadmap for the proof of the upper bound. We consider the invariant cone

$$\mathcal{C} := \left\{ \begin{pmatrix} a \\ b \end{pmatrix} : a \geq b \geq 0 \right\},$$

such that every EEA transformer matrix $M(q)$ preserves $\mathcal{C}$, since $M(q)(a,b)^T = (qa+b,a)^T \in \mathcal{C}$ for all $q \geq 1, a \geq b \geq 0$.

To derive the minimum possible value of p, we use the toolkit of a finite polyhedral antinorm certificate on the invariant cone $\mathcal{C}$, following the extremal-antinorm method for lower spectral radius from [GP13]. An antinorm on an invariant cone is a non-zero, non-negative, concave, and positively homogeneous function. It is positive everywhere inside the cone, except at the origin. In particular, the minimum of finitely many positive linear functions, $F(v) = \min_{u \in U} uv$ for some (positive-coordinated) vector set U, is a positive polyhedral antinorm.

Set $\rho = 2+\sqrt{3}, \lambda = \rho^{1/3} = (2+\sqrt{3})^{1/3}$ as discussed in the asymptotic sharpness of the upper bound. We want to find an antinorm $F = \min_{u \in U} uv$ with the property that the inequality $F(M(q)v) \geq \lambda^{d(q)} F(v)$ holds for every $q \geq 1$ and every v in the invariant cone $\mathcal{C}$. Iterating this inequality converts the total weight $N/4$ directly into the lower growth factor $\lambda^{N/4}$, which lower-bounds the possible value of p.

For $d \geq 0$, let $B_d := \lambda^{-(d+1)} M(2^d)$ be the normalized transformation associated with the smallest quotient $q = 2^{d-1}$ of bit-length $(d+1)$. To achieve the inequality $F(M(q)v) \geq \lambda^{d(q)} F(v)$ for $d \leq 4$, we need to construct a finite set $U = \{u_0, \ldots, u_r\}$ such that

$$u_j B_d v \geq \min_{u \in U} uv = F(v), \text{ for all } u_j \in U, d \in \{0,1,2,3\} \text{ and } v \in \mathcal{C}. \tag{12}$$

Once the condition (12) is satisfied, we have $F(B_d v) \geq F(v)$, therefore $F(M(2^d)v) \geq \lambda^{d+1} F(v)$.

The first two vectors of U encode the two-step return generated by the repeated block $(1,2)$. Namely, we choose u_0 to be the left Perron eigenvector of $M = M(1)M(2)$. The $(1,2)$ cycle determines $u_1 = \lambda^{-1} u_0 M(1)$ and gives $u_0 = \lambda^{-2} u_1 M(2)$. Thus, u_0 and u_1 are precisely the two supporting directions visited during one period of the optimal quotient sequence.

However, although the two vectors explain the candidate periodic pattern, they do not yet give a certificate for arbitrary quotients. For a current row set U, we say that a transformed row $w = uB_d$ is already controlled by the antinorm $F_U(v) := \min_{u' \in U} u'v$ if there is some $h \in \mathrm{conv}(U)$ such that $wv \geq hv$ for every $v \in \mathcal{C}$. Indeed, this implies $wv \geq hv \geq \min_{u' \in U} u'v = F_U(v)$.

Starting from $U = \{u_0, u_1\}$, the transformation B_2, corresponding to the quotient $q = 4$, produces $u_2 := u_1 B_2 = \lambda^{-3} u_1 M(4)$. This row is not controlled by the two-row antinorm as $u_2(1,1)^T < \min_{0 \leq j \leq 1} u_j(1,1)^T$, thus must be added to U. Applying B_0 to the new row similarly gives $u_3 := u_2 B_0 = \lambda^{-1} u_2 M(1)$, and $u_3(1,0)^T < \min_{0 \leq j \leq 2} u_j(1,0)^T$, so a fourth row is required for U. For the resulting set $U = \{u_0, u_1, u_2, u_3\}$, we can verify that the row $u_j B_s$ dominates on $\mathcal{C}$ a row in $\mathrm{conv}(U)$, for every $j, s \in \{0,1,2,3\}$. Hence all normalized transformations B_s associated with bit-lengths at most four are controlled, and the finite row-generation procedure stops.

For the case $d \geq 5$, the inequality $F(M(2^d)v) \geq \lambda^{d+1} F(v)$ are not handled by this finite closure table, but instead covered by the coarser estimate $F(M(q)v) \geq (q/\lambda) F(v) \geq \lambda^{d+1} F(v)$. Since $q \geq 2^{d-1} \geq \lambda^{d+1}$ holds for all $d \geq 5$.

Formal proof of the upper bound. Set $\rho = 2+\sqrt{3}, \lambda = \rho^{1/3} = (2+\sqrt{3})^{1/3}$ and $\mathcal{C} = \{(a,b)^T : a \geq b \geq 0\}$. Define the following four positive row vectors:

$$u_0 := (1+\sqrt{3}, 1), \qquad u_1 := \lambda^{-1} u_0 M(1), \qquad u_2 := \lambda^{-3} u_1 M(4), \qquad u_3 := \lambda^{-1} u_2 M(1).$$

Equivalently, using $\lambda^3 - 1 = 1 + \sqrt{3}$,

$$u_0 = (\lambda^3 - 1, 1), \quad u_1 = \left(\lambda^2, \frac{\lambda^3-1}{\lambda}\right), \quad u_2 = \left(\frac{5}{\lambda} - \frac{1}{\lambda^4}, \frac{1}{\lambda}\right), \quad u_3 = \left(\frac{6\lambda^3-1}{\lambda^5}, \frac{5\lambda^3-1}{\lambda^5}\right).$$

For $v \in \mathcal{C}$, define

$$F(v) := \min_{0 \leq j \leq 3} u_j v.$$

Since the coordinates of the four vectors are positive, it is easy to prove that the function F is positive on $\mathcal{C} \setminus \{0\}$, homogeneous, concave, and coordinatewise nondecreasing. For the function F, we have the following lemma.

Lemma A.1. *Let $q \geq 1$ be a positive integer, $d = \lfloor \log_2 q \rfloor + 1$. For any vector $v \in \mathcal{C}$, we have*

$$F(M(q)v) \geq \lambda^d F(v). \tag{13}$$

Moreover, if $d \geq 2$, then

$$F\left((q,1)^T\right) \geq \lambda^{d-2} F\left((2,1)^T\right). \tag{14}$$

The overall upper bound of EEA step number follows directly from Lemma A.1. More precisely, we have that

$$\begin{aligned}
\frac{p}{2} \cdot F\left((2,1)^T\right) &= F\left((p,p/2)^T\right) \geq F\left((p,x)^T\right) && \text{(By Algorithm 1, } x < p/2\text{)} \\
&= F\left(M(q_1)\cdots M(q_{m-1}) \begin{pmatrix} q_m \\ 1 \end{pmatrix}\right) && \text{(By Equation 10)} \\
&\geq \lambda^{N/4-d_m} F\left((q_m,1)^T\right) && \text{(By Inequality 13)} \\
&\geq \lambda^{N/4-2} F\left((2,1)^T\right). && \text{(By Inequality 14, where } r_{m-1} \geq 2 \text{ implies } d_m \geq 2\text{)}
\end{aligned}$$

This implies that

$$N/4 \leq 2 + \frac{\log_2 p - 1}{\log_2 \lambda} < 2 + \frac{n-1}{\log_2 \lambda} < 1 + \frac{n}{\log_2 \lambda}.$$

As $N/4$ is an integer, we can conclude that $N \leq 4\lceil cn \rceil$, where $c = 1/\log_2 \lambda = 3/\log_2(2+\sqrt{3})$.

What left is to prove Lemma A.1.

Proof of Lemma A.1. For row vectors a, b, write $a \succeq_{\mathcal{C}} b$ if $(a-b)(1,0)^T \geq 0$ and $(a-b)(1,1)^T \geq 0$. Since each element in $\mathcal{C}$ can be represented as a linear combination of $(1,0)^T$ and $(1,1)^T$ with nonnegative coefficients, the relation $a \succeq_{\mathcal{C}} b$ implies $av \geq bv$ for every $v \in \mathcal{C}$.

We first prove (13). Define the matrix $G = (G_{j,s})$ for $j = 0, \ldots, 3$ and $s = 0, \ldots, 3$ by

$$G_{j,s} := \lambda^{-s-1} u_j M(2^s).$$

and the matrix $H = (H_{j,s})$ with entries

$$H = \begin{pmatrix} u_1 & \frac{4}{5}u_0 + \frac{1}{5}u_1 & u_0 & u_0 \\ u_1 & u_0 & u_2 & u_0 \\ u_3 & u_0 & u_0 & u_0 \\ u_0 & u_0 & u_2 & u_0 \end{pmatrix}.$$

It is trivial to verify that

$$G_{j,s} \succeq_{\mathcal{C}} H_{j,s} \quad \text{and} \quad H_{j,s}v \geq F(v), \quad \text{for all } j, s \in \{0, ..., 3\}.$$

Consequently,

$$F(M(2^d)v) = \lambda^{d+1} \min_{0 \leq j \leq 3} G_{j,d}v \geq \lambda^{d+1} F(v), \quad \text{for } 0 \leq d \leq 3.$$

Since F is coordinatewise nondecreasing, $F(M(q)v) \geq F(M(2^{d-1})v) \geq \lambda^d F(v)$. Thus we prove (13) for $1 \leq d \leq 4$.

It remains to consider $d \geq 5$. Write $v = (a, b)^T, a \geq b \geq 0$. A direct check of the four defining rows gives $F((1,0)^T) = u_3(1,0)^T, F((1,1)^T) = u_2(1,1)^T = \lambda F((1,0)^T)$. By monotonicity and homogeneity,

$$F(M(q)v) = F\begin{pmatrix} qa + b \\ a \end{pmatrix} \geq F\begin{pmatrix} qa \\ 0 \end{pmatrix} = qaF\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{qa}{\lambda} F\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{q}{\lambda} F\begin{pmatrix} a \\ a \end{pmatrix} \geq \frac{q}{\lambda} F(v). \tag{15}$$

For $d = 5$, $2^4 = 16 > \lambda^6 = (2 + \sqrt{3})^2 = 7 + 4\sqrt{3}$, and the inequality propagates to all larger d, since $2 > \lambda$. Hence $q/\lambda \geq 2^{d-1}/\lambda \geq \lambda^d$. Together with (15) this proves (13) for every $d \geq 5$.

We next prove (14). Since $q \geq 2^{d-1}$, it suffices by monotonicity to consider $q = 2^{d-1}$:

- For $d = 2$, the claim is immediate.
- For $d = 3$, direct substitution gives $F((4,1)^T) \geq \lambda F((2,1)^T)$.
- For $d \geq 4$, the same four-row calculation gives $8F(1,0)^T \geq \lambda^2 F(2,1)^T$. Therefore,

$$F\begin{pmatrix} 2^{d-1} \\ 1 \end{pmatrix} \geq 2^{d-1} F\begin{pmatrix} 1 \\ 0 \end{pmatrix} = 2^{d-4}\left(8F\begin{pmatrix} 1 \\ 0 \end{pmatrix}\right) \geq 2^{d-4}\lambda^2 F\begin{pmatrix} 2 \\ 1 \end{pmatrix} \geq \lambda^{d-2} F\begin{pmatrix} 2 \\ 1 \end{pmatrix},$$

  where the last inequality uses $2 > \lambda$.

This completes the proof. □

A.2 Step-dependent active windows

Fix an input x, let $(q_1, \ldots, q_{k-1})$ denote the quotient sequence produced by EEA, and define $b_i := \lfloor \log_2 q_i \rfloor$ for each iteration i. Under the four-phase schedule, each EEA iteration i is expanded into $4(b_i + 1)$ steps. The total number of steps is $N = 4\sum_{i=1}^{k-1}(b_i + 1) \leq N_{\max} := 4\lceil cn \rceil$, where $\lambda := (2 + \sqrt{3})^{1/3}$ and $c := 1/\log_2 \lambda = 3/\log_2(2 + \sqrt{3})$ (see Appendix A.1).

For a global step index $T \in \{1, 2, \ldots, N_{\max}\}$, let $j = j(x, T)$ denote the index of the EEA iteration containing step T, and let $u = u(x, T)$ denote the position of T within that iteration. More precisely, the pair $(j(x, T), u(x, T))$ is uniquely determined by the conditions

$$\sum_{i=1}^{j-1} 4(b_i + 1) < T \le \sum_{i=1}^{j} 4(b_i + 1), \quad u = T - \sum_{i=1}^{j-1} 4(b_i + 1).$$

The bit-length of the current quotient $\ell_q(x, T)$ depends deterministically on indices (j, u):

$$\ell_q(x, T) = \begin{cases} 0 & u(x, T) \in (0, b_j + 1] \cup (3(b_j + 1), 4(b_j + 1)], \\ u(x, T) - (b_j + 1) & u(x, T) \in (b_j + 1, 2(b_j + 1)], \\ 3(b_j + 1) - u & u(x, T) \in (2(b_j + 1), 3(b_j + 1)]. \end{cases} \tag{16}$$

We next state a lemma that characterizes the minimal possible value of t_j, conditioned on the total number of steps N_j taken before reaching the j-th iteration of EEA. Let F be the function constructed in Appendix A.1, and define

$$\eta := \frac{F\left((2,1)^T\right)}{\lambda^2 F\left((1,1)^T\right)} = \frac{4\sqrt{3} - 3}{13}\lambda^2, \qquad \delta := -\log_\lambda \eta = 0.726208\ldots.$$

Lemma A.2. *Suppose that $N_j = \sum_{i=1}^{j-1} 4(b_i + 1)$ is fixed. Then, over all possible EEA iteration indices j and all admissible quotient sequences $(q_1, \ldots, q_{j-1})$, we have $t_j \ge \eta\lambda^{N_j/4} = \lambda^{N_j/4-\delta}$. Consequently,*

$$\lfloor \log_2 t_j \rfloor + 1 > \frac{N_j/4 - \delta}{c} = \frac{N_j - 4\delta}{4c}, \qquad \lfloor \log_2 t_j \rfloor + 1 \ge \left\lceil \frac{N_j - 4\delta}{4c} \right\rceil.$$

Proof. By Algorithm 1, the EEA input x is chosen to be smaller than $p/2$, and hence $q_1 \ge 2$. Set $d_i := b_i + 1$. The Lemma A.1 established in Appendix A.1 give $F(M(q)v) \ge \lambda^{\lfloor \log_2 q \rfloor + 1} F(v)$ and $F\left((q,1)^T\right) \ge \lambda^{d-2} F\left((2,1)^T\right)$, whenever $q \ge 2$ has bit-length d. Given the EEA iteration rule $t_{i+1} = t_{i-1} + q_i t_i$, we can write that $(t_{i+1}, t_i)^T = M(q_i)(t_i, t_{i-1})^t$. Therefore, for $j \ge 2$,

$$F\left((t_j, t_{j-1})^T\right) = F\left(M(q_{j-1}) \cdots M(q_2) \begin{pmatrix} q_1 \\ 1 \end{pmatrix}\right) \ge \lambda^{N_j/4 - d_1} F\left((q_1, 1)^T\right) \ge \lambda^{N_j/4-2} F\left((2,1)^T\right).$$

Since $0 \le t_{j-1} \le t_j$ and F is coordinatewise nondecreasing and positively homogeneous,

$$F\left((t_j, t_{j-1})^T\right) \le F\left((t_j, t_j)^T\right) = t_j F\left((1,1)^T\right).$$

Combining the two inequalities gives $t_j \ge \eta\lambda^{N_j/4}$. The case $j = 1$ follows from $t_1 = 1 \ge \eta$. □

We then provide the detailed proofs of the bounds stated in Section 4.2 in the remainder of this subsection.

Location-controlled addition/subtraction on r's. First, observe that $t_j q_j \le t_{j+1} < p$, which implies $\ell_t + b_j \le n+1$. Since a location-controlled addition/subtraction on r's is only activated in Phase 1 and 2, i.e. $0 < u \le 2(b_j+1)$, we then analyze the claimed bound by considering two cases.

- **Case 1.** Suppose that $u \le b_j + 1$. In this case, we have $\ell_q = 0$. By Lemma A.2, it follows that
$$\ell_t > \frac{T-u-4\delta}{4c} \ge \frac{T-(n+1-\ell_t)-4\delta}{4c}.$$
Hence $\ell_t + \ell_q = \ell_t > \dfrac{T-n-1-4\delta}{4c-1}$.
- **Case 2.** Suppose that $b_j + 1 < u \le 2(b_j+1)$. Then $\ell_q = u - (b_j+1)$, and we obtain that
$$\ell_t + \ell_q > \frac{T-u-4\delta}{4c} + u - (b_j+1).$$
The right-hand side is minimized when $u = b_j + 1$, in which case the expression reduces to that of Case 1.

Combining the two cases, we conclude that $\ell_t + \ell_q + 2 \ge \max\left\{\left\lceil \dfrac{T-n-1-4\delta}{4c-1}\right\rceil, 1\right\} + 2$.

Location-controlled swaps. For the lower bound, a location-controlled swap is only activated in Phase 2 and 3, i.e. $b_j + 1 < u \le 3(b_j+1)$, we then analyze the claimed lower bound by considering two cases.

- **Case 1.** Suppose that $b_j + 1 < u \le 2(b_j+1)$. In this case, we also have $\ell_t + \ell_q \ge \dfrac{T-n-1-4\delta}{4c-1}$.
- **Case 2.** Suppose that $2(b_j+1) < u \le 3(b_j+1)$. In this case, we have $\ell_q = 3(b_j+1) - u$. By Lemma A.2, it follows that
$$\ell_t \ge \frac{T-u-4\delta}{4c} \ge \frac{T-3(n+1-\ell_t)-4\delta}{4c},$$
hence $\ell_t + \ell_q \ge \ell_t \ge \dfrac{T-3(n+1)-4\delta}{4c-3}$.

The Case 1 bound is no smaller than the Case 2 bound throughout the active range. Indeed, their difference equals $\dfrac{2(4cn+4c+4\delta-T)}{(4c-1)(4c-3)} > 0$, where we used $T \le N_{\max} < 4cn+4$. Combining the two cases, we conclude that $\ell_t + \ell_q + 1 \ge \max\left\{\left\lceil \dfrac{T-3(n+1)-4\delta}{4c-3}\right\rceil, 1\right\} + 1$.

For the upper bound, recall the recurrence relation of the EEA on the t-sequence, that is $t_{i+1} = t_{i-1} + q_i t_i < (q_i+1)t_i \le 2^{b_i+1} t_i$. By iterating this inequality, we obtain that
$$t = t_j \le \prod_{i=1}^{j-1} 2^{b_i+1} = 2^{(T-u)/4}, \quad \text{and then} \quad \ell_t \le (T-u)/4 + 1.$$
By Equation (16), we know that $\ell_q \le u/2$. Hence,
$$\ell_t + \ell_q + 1 \le \lfloor (T-u)/4 + u/2 + 2 \rfloor = \lfloor (T+u)/4 \rfloor + 2 \le \lfloor T/2 \rfloor + 2.$$
Finally, since the `Work1` register stores a value $r \ge 1$, we conclude that $\ell_t + \ell_q + 1 \le n+2$.

Location-controlled addition/subtraction on t's. As shown above, we have that $\ell_t+1 \le \lfloor (T-u)/4 \rfloor + 2 \le \lceil T/4 \rceil + 1$. Under the constraint that $t \le p < 2^n$, we also obtain that $\ell_t + 1 \le n+1$. Therefore, $\ell_t + 1 \le \min\{\lceil T/4 \rceil + 1, n+1\}$.

Length-update circuit. For the lower bound $k_4(T)$, the length-update circuit is activated only at the end of an EEA iteration, namely when $u = 4(b_j+1)$. By Lemma A.2, we have

$$\ell_t \ge \frac{T-u-4\delta}{4c} \ge \frac{T-4(n+1-\ell_t)-4\delta}{4c},$$

which implies $\ell_t \ge \dfrac{T-4(n+1)-4\delta}{4c-4}$.

For the lower bound $k_5(T)$, Lemma A.2 directly yields $\ell_t^* \ge \dfrac{T-4\delta}{4c}$. For the upper bound $K_5(T)$, recall the recurrence relation of the EEA on the r-sequence, $r_{i-1} = r_{i+1} + q_i r_i < (q_i+1) r_i \le 2^{b_i+1} r_i$. By iterating this inequality, we obtain

$$p/r_{j+1} \le \prod_{i=1}^{j} 2^{b_i+1} = 2^{T/4}.$$

It follows that $n+4-\ell_{r'}^* < n+4-\log_2 r_{j+1} \le T/4+n+4-\log_2 p \le T/4+5$, and hence $n+4-\ell_{r'}^* \le T/4+4$.

For the upper bound $K_4(T)$, we similarly have $n+4-\ell_{r'} \le n+4-\ell_{r'}^* \le T/4+4$.

B Improved Modular Multiplication Circuit

Our modular multiplier used in in-place division/multiplication implements

$$\mathrm{Mul}_p : |x\rangle|y\rangle|0\rangle \longmapsto |x\rangle|y\rangle|xy \bmod p\rangle,$$

based on the usual Horner double-and-add scan as the low-space multiplier of [RNSL17]. The circuit contains n controlled modular additions and $n-1$ modular doublings.

- **Controlled modular additions.** The controlled modular addition

  $$|c\rangle|A\rangle|B\rangle|0\rangle_f \longmapsto |c\rangle|A\rangle|B + cA \bmod p\rangle|0\rangle_f,$$

  is implemented by first doing the controlled addition $B \leftarrow B + cA \pmod{2^n}$, while XORing the overflow carry into a flag f. We use a specialized controlled quantum–quantum ripple-carry adder for this step, rather than controlling every gate of an ordinary adder from [CDKM04]. Let γ_i denote the carry into bit i, with $\gamma_0 = 0$, and $\gamma_{i+1} = \mathrm{maj}(A_i, B_i, \gamma_i)$.

  During the forward sweep, the circuit computes the carry chain. At bit i, the addend and accumulator wires are temporarily transformed to $A_i \oplus \gamma_i$ and $B_i \oplus \gamma_i$, while the carry wire is advanced to γ_{i+1}. In the reverse sweep, we first undo the carry Toffoli, so the carry wire again contains γ_i. A CNOT from this carry wire to the accumulator wire restores $B_i \oplus \gamma_i \longmapsto B_i$. At this point the addend wire still contains $A_i \oplus \gamma_i$. We then apply the controlled sum-writing Toffoli

  $$\mathrm{CCX}\big(c,\ A_i \oplus \gamma_i;\ B_i\big).$$

Thus, if $c = 0$, the accumulator bit remains B_i; if $c = 1$, it becomes $B_i \oplus A_i \oplus \gamma_i$, which is exactly the i-th sum bit. Finally, a CNOT from the carry wire to the addend wire restores $A_i \oplus \gamma_i \longmapsto A_i$. Therefore, the controlled writing of the n sum bits contributes precisely n additional Toffoli gates. In addition, after the forward sweep the final carry γ_n is copied into the overflow flag only when $c = 1$, using one more CCX gate,

$$f \longleftarrow f \oplus c\gamma_n.$$

Hence this controlled variant of ripple-carry adder costs $3n + O(1)$ Toffoli gates.

Let $S = B + cA$, then after the first step the accumulator contains

$$\widetilde{B} = S \bmod 2^n,$$

and the overflow bit

$$h = \lfloor S/2^n \rfloor$$

has been XORed into the flag. Since $A, B_{\text{old}} < p < 2^n$, we have $S \leq 2p - 2 < 2^{n+1}$, so $h \in \{0, 1\}$. The true reduction condition is $S \geq p$. If $h = 0$, then $\widetilde{B} = S$, so this condition is exactly $[\widetilde{B} \geq p]$. If $h = 1$, then reduction is certainly needed, while $\widetilde{B} = S - 2^n < p$, so $[\widetilde{B} \geq p] = 0$. Hence the reduction bit is

$$r = h \oplus [\widetilde{B} \geq p].$$

We therefore update the flag by XORing in the fixed-modulus comparison $[\widetilde{B} \geq p]$, which is evaluated by the classical-quantum comparator from [Gid25]. Here we use the dirty-assisted adder which costs $3n+O(1)$ Toffoli gates by borrowing the non-target register A as dirty workspace. Controlled on this flag, we subtract the fixed modulus p from B, again using the same Gidney-style controlled classical–quantum subtractor, with cost $3n + O(1)$. Finally, after the correction, the flag can be uncomputed by one Toffoli gate applying

$$f \leftarrow f \oplus c\,[B_{\text{final}} < A].$$

$[B_{\text{final}} < A]$ can be evaluated using the high-bit-only Cuccaro quantum comparator, derived from the Cuccaro ripple-carry adder [CDKM04], costing $2n+O(1)$ Toffoli gates. All clean and borrowed dirty workspace, including the flag f, is restored at the end of the block.

Therefore, one controlled modular addition costs

$$T_{\text{add}}(n) = (3n + O(1)) + (3n + O(1)) + (3n + O(1)) + (2n + O(1)) = 11n + O(1)$$

Toffoli gates.

- **Modular doublings.** For modular doubling, we follow the implementation from [RNSL17]. The binary doubling itself is implemented by Clifford shift/CNOT operations. Since p is odd, the reduction flag can be cleared from the least significant output bit. The only parts that require Toffoli gates are one fixed comparison with p and one conditional fixed subtraction, which can be implemented by the same classical-quantum adder from [Gid25].

  Therefore, one modular doubling costs

$$T_{\text{dbl}}(n) = (3n + O(1)) + (3n + O(1)) = 6n + O(1)$$

  Toffoli gates.

Consequently, the total number of Toffoli gates required by our modular multiplication circuit grows asymptotically as

$$T(n) = n\,T_{\mathrm{add}}(n) + (n-1)\,T_{\mathrm{dbl}}(n) = 17n^2 + O(n).$$

The modular squaring circuit is same to the modular multiplication, but with the addend equal to the input register itself. Therefore, it also costs at most $17n^2 + O(n)$ Toffoli gates.